Universidade Federal da Paraíba

Programa de Pós-Graduação em Informática

Altitude Inference Over Water Surface Using Convolutional Neural Networks

João Vitor da Silva Neto

João Pessoa - PB

2026

Universidade Federal da Paraíba

Programa de Pós-Graduação em Informática

João Vitor da Silva Neto

Altitude Inference Over Water Surface Using Convolutional Neural Networks

Dissertação apresentada à Universidade Federal da Paraíba como parte dos requisitos para a obtenção do título de Mestre em Informática.

Orientador: Prof. Dr. Tiago Pereira do Nascimento

Coorientador: Prof. Dr. Leonardo Vidal Batista

João Pessoa - PB

2026

Ficha catalográfica: elaborada pela biblioteca do CI.

Será impressa no verso da folha de rosto e não deverá ser contada.
Se não houver biblioteca, deixar em branco.

Está página deverá ser substituída por uma folha contendo as assinaturas dos membros da banca, e deve ser colocada após a ficha catalográfica

Abstract

The need for redundant odometry from a wide variety of sensing sources benefits from recent advances in fast and efficient real-time image processing through classification and linear regression algorithms. This study proposes an approach based on Convolutional Neural Networks (CNNs) for numerically estimating the distance between the imaging sensor and the surveyed surface during UAV operations over water. The CNN system was trained on synthetic images generated in Blender, and real images captured by a remotely controlled quadrotor were evaluated; in addition, the associated synchronized odometry data were collected for comparison. Preliminary tests demonstrated the system's ability to recognize different distances from the water surface, even when the real-image dataset was under non-ideal conditions, in an uncontrolled environment with dirt and shadows, after the appropriate processing and filtering described in the scope of the work.

Keywords: UAV, offshore, machine learning, height estimation

Resumo

A necessidade de odometria redundante com as mais diversas fontes de captação, se beneficia das recentes possibilidades de processamento rápido e eficiente da imagem em tempo real, por meio de algoritmos de classificação e regressão linear. O presente trabalho propõe uma abordagem baseada em Redes Neurais Convolutivas (RNC), utilizadas para detectar numericamente a distância entre o sensores de imagem e a superfície sobrevoada durante a operação de VANTs sobre a água. O sistema de RNC foi treinado com imagens sintéticas geradas em Blender, e avaliaram imagens reais capturadas por um quadrotor controlado remotamente, coletando, também, os dados de odometria inerentes e sincronizados para respectiva comparação. Os testes preliminares demonstraram a capacidade do sistema de reconhecer diferentes distâncias da superfície da água, mesmo com o conjunto de imagens reais em condição não ótima, em ambiente não controlado, com sujeiras e sombras, após o devido tratamento e filtragem descritos no escopo do trabalho.

Palavras-chave: VANT, offshore, aprendizagem de máquina, estimação de altitude

Acknowledgement

Not even a single line of this work would have been possible without the help of people for whom I hold deep respect and admiration, both in my academic and personal journey.

I thank my parents, Antônio Vitor and Danúbia Lins, for their unconditional support throughout every phase of doubt and belief I have gone through. I also thank Professor PhD. Tiago Pereira do Nascimento for standing by me and for always helping me find paths in my professional and research journey. To Professor PhD. Leonardo Vidal Batista, for the many discussions about methods, from whom I learned much of what I will continue applying to the solutions of the problems I face in my day-to-day work as a developer.

I thank my wife, Kamila Alves, for her patience and understanding throughout the many long work trips that kept us apart. To the excellent friends I made in the lab and through my work, who provided me with unusual ideas and time to unwind, I will carry with me for the rest of my life. And, above all, I thank God for giving me answers in the many moments when everything seemed to go wrong.

"Para tudo há um tempo determinado; Há um tempo para toda atividade debaixo dos céus [...]"

– Eclesiastes 3:1 TNM

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS

CPU - Central Processing Unit

CNN - Convolutional Neural Network

GPS - Global Positioning System

GPU - Graphics Computing Unit

LIDAR - Light Detection And Ranging

MAE - Mean Absolute Error

MAPE - Mean Absolute Percentage Error

MAV - Micro Aerial Vehicle

MMTM - Multimodal Transfer Module

PSF - Point Spread Function

RADAR - Radio Detection And Ranging

ReLU - Rectified Linear Unit

RMSE - Root Mean Squared Error

TF - TensorFlow

UAV - Unmanned Aerial Vehicle

UVDAR - Ultraviolet Direction and Ranging

Contents

1 INTRODUCTION

1.1 Problem Description

Accurate altitude estimation is a fundamental requirement for the safe operation of aerial vehicles, from large fixed-wing aircraft to the simplest lightweight quad rotors. In most flight control systems, this estimation relies on a combination of onboard sensors whose complementary characteristics and built-in redundancies are fused through mathematical filtering techniques, such as Kalman filters and complementary filters, to produce reliable altitude measurements.

However, achieving safe and autonomous flight depends not only on sensor quality but also on the compatibility between the operating environment and the sensor suite carried by the vehicle. Terrain characteristics, atmospheric conditions, and surface properties all influence how well a given sensor can perform. Consequently, a sensor that delivers excellent accuracy in one scenario may become unreliable or entirely non-functional in another.

Traditionally, altitude estimation in unmanned aerial vehicle (UAV) flight controllers is accomplished by fusing measurements from a variety of sensing modalities. Light Detection and Ranging (LIDAR) provides high-resolution distance measurements through laser pulses; ultrasonic rangefinders offer short-range proximity detection through acoustic echo timing; stereo imaging systems reconstruct depth from binocular disparity; barometric altimeters infer altitude from atmospheric pressure differentials; and classical computer vision algorithms estimate distance by comparing known structural dimensions against their apparent size in the image plane. Ultraviolet Direction and Ranging (UVDAR) systems, originally developed for mutual localization among UAV swarms, further complement these approaches by using ultraviolet markers for relative positioning. Each of these methods is well-established and has demonstrated robust performance in conventional terrestrial environments.

Despite their maturity, all of these sensing modalities share a critical limitation: they degrade substantially or fail entirely when operating over water surfaces. Water presents a uniquely challenging terrain for altitude sensing due to several interacting physical phenomena. Its surface is in constant motion driven by wind, currents, and thermal convection, creating an irregular and time-varying geometry that prevents ultrasonic sensors from receiving consistent echo returns. The specular and semi-transparent optical properties of water, combined with surface disturbances such as ripples, waves, and foam, severely impair light-based sensors including LIDAR, infrared rangefinders, and UVDAR systems, which depend on predictable surface reflections to compute range. Furthermore,

the barometric altimeter, often considered a reliable fallback, suffers from significant drift when operating near water due to localized pressure variations caused by evaporation, humidity gradients, and the high thermal capacity of large water bodies, all of which introduce density changes in the air column above the surface.

These limitations pose a significant practical problem. As UAV applications expand into maritime inspection, environmental monitoring of rivers and lakes, offshore infrastructure maintenance, and search-and-rescue operations over open water, the inability to obtain reliable altitude measurements over aquatic surfaces becomes a critical bottleneck for safe autonomous flight.[57]

With the rapid advancement of real-time image processing technologies and the increasing availability of embedded computing platforms capable of running deep learning models, a paradigm shift has been observed across many engineering domains: machine learning approaches are progressively assisting, complementing, and in some cases replacing classical numerical methods in sensor processing pipelines [17]. This trend opens the possibility of using visual information alone, captured by a downward-facing camera, to infer altitude above surfaces where conventional sensors fail.

The specific challenge addressed in this work, altitude estimation above water using monocular imagery, demands exploration of methods that fall outside the conventional sensor fusion framework. It requires a deeper understanding of the physical properties that make water surfaces unsuitable for measurement by existing equipment, as well as the identification of visual cues in images of water that correlate with altitude [73]. The surface texture scale, brightness patterns, and the spatial frequency of visible features all change sufficiently predictably with distance, providing a signal that, while subtle to formalize analytically, can be learned by a convolutional neural network trained on appropriately constructed datasets with 3D software like Blender or Unreal Engine 4 [36].

1.2 Objectives

The primary objective of this work is to propose and evaluate an image-based altitude estimation system for UAV operation over water surfaces, using a convolutional neural network trained to infer based on regression, targeting the distance from monocular images captured by a downward-facing camera. The secondary objectives are:

- Photorealistic modeling of water surfaces for synthetic data generation
- Creation of a large-scale (190k images) synthetic image database spanning multiple altitude levels
- Construction of an initial real-world database for cross-domain validation

- Training, evaluation, and benchmarking of CNN regression systems to achieve the primary objective

1.3 Hypothesis

The central hypothesis of this work is that, in the same way humans can visually estimate their proximity to a surface without any instrumentation, a machine learning model can be trained to extract and interpret the visual cues that encode distance information in images of water. For this hypothesis to hold, the synthetic images used for training must be recognizable by human observers as realistic representations of water, and they must exhibit visually distinguishable differences in appearance as a function of rendering altitude.

If this hypothesis is validated, the resulting system can be embedded in UAV platforms as a complementary altitude sensing modality. The model also needs to match the specific hardware capabilities to be held on, not excluding the need for quantization methods or even a reduced number of layers and neurons. Beyond mobile robotics, such a capability could be extended and adapted to other domains where visual distance estimation over water is relevant, such as oceanographic surveying, coastal monitoring, and autonomous maritime navigation.

1.4 Structure of This Work

Chapter 2 provides the theoretical foundations underlying this work, divided into two main topics: an overview of convolutional neural networks and their learning mechanisms in Subsection 2.1, and a summary of altitude estimation methods and the sensors commonly employed in UAV systems in Subsection 2.2.

Chapter 3 presents a review of the related literature, covering the most influential works in computer vision-based altitude estimation, as well as supporting references on the optical and physical properties of water that inform the design decisions made throughout this study.

Chapter 4 describes the methodology, organized into six subsections for clarity. Subsection 4.1 details the generation of synthetic water surface images using the 3D modeling software Blender, along with the preprocessing pipeline implemented in OpenCV. Subsections 4.2 and 4.3 cover the acquisition of the real-world dataset, including equipment specifications, characteristics of the measurement site, and the data normalization procedures applied. Subsection 4.4 describes the computational platform with its hardware and software specifications. Subsection 4.5 presents the neural network architecture,

including details on activation functions and model parameters. Finally, Subsection 4.6 discusses the optimization algorithms used during the training process.

Chapter 5 presents and discusses the results, including performance metrics and visualizations for both the synthetic training set and the real-world evaluation set, along with a comparative analysis of the optimizers employed.

Chapter 6 outlines the planned work for the upcoming months, including a projected schedule and a description of the remaining tasks.

2 THEORETICAL FOUNDATION

This chapter presents the theoretical background underlying the methods employed in this work. It begins with the physics and mathematics of water surface modeling, from classical wave theory through modern computer graphics implementations. Then it addresses the image degradation introduced by quadcopter motion, followed by the feature engineering techniques used to extract altitude-dependent information from water surface images. The chapter concludes with the machine learning architectures and evaluation metrics used for altitude regression.

2.1 Water Surface Modeling

The visual appearance of a water surface as seen from a downward-facing camera is fundamentally determined by the geometry of the surface waves, the optical properties of the air-water interface, and the illumination conditions. Generating photorealistic synthetic images of water at controlled altitudes requires a physically grounded model of surface wave dynamics. This subsection traces the development of such models from their analytical origins to their implementation in modern rendering software.

2.1.1 First Models

The mathematical description of water surface waves has its origins in the work of Sir George Biddell Airy, who in 1841 formulated what is now known as linear wave theory [2]. Airy's model assumes a zero-viscosity, non-rotational, incompressible fluid of uniform depth, and describes the surface elevation as a sinusoidal function of position and time:

$$\eta(x,t) = A\cos(kx - \omega t) \tag{1}$$

Where A is the wave amplitude, $k = 2\pi/\lambda$ is the wavenumber, λ is the wavelength, and ω is the angular frequency. The key result of linear theory is the *dispersion relation*, which connects frequency to wavenumber through gravity

$$\omega^2 = gk\tanh(kD) \tag{2}$$

where g is the gravitational acceleration and D is the water depth. In deep water ($kD \gg 1$), this simplifies to $\omega^2 = gk$, meaning longer waves travel faster than shorter ones, defining the property of dispersive wave propagation. The linearity of Airy's model means that individual wave components can be superimposed without interaction, which is the theoretical foundation enabling spectral synthesis methods.

Stokes extended Airy's framework in 1847 with a perturbation expansion that accounts for nonlinear effects [58]. The Stokes wave theory introduces corrections for finite wave steepness: wave crests become sharper and troughs flatter relative to the sinusoidal profile, wave crests propagate slightly faster than linear theory predicts, and particles exhibit a net drift in the direction of wave propagation (Stokes drift). These nonlinear corrections become significant when the wave steepness kA approaches unity, which occurs in energetic sea states and near breaking conditions.

The transition from deterministic wave equations to statistical descriptions of the ocean surface was driven by the recognition that real sea states consist of many superimposed wave components with different frequencies, directions, and phases. Phillips derived, through dimensional analysis, that the high-frequency tail of the wind-wave energy spectrum follows a universal form

$$\Phi(\omega) = \alpha g^2 \omega^{-5} \tag{3}$$

where $\alpha \approx 8.1 \times 10^{-3}$ is the Phillips constant [45]. This ω^{-5} decay arises from the equilibrium between wind energy input and dissipation through wave break, and is independent of wind speed, a powerful result that constrains the spectral shape at high frequencies.

A comprehensive review of the mathematical development from Newton through Stokes to modern spectral theory is provided by Craik [11], while Darles et al. [14] survey the adoption of these models in computer graphics rendering pipelines, and raise a critical insight that connects spectral wave theory to synthetic image generation: that linear wave theory permits the ocean surface elevation to be expressed as a superposition of sinusoidal components, exactly the form compatible with Fourier synthesis. The spectral models described above provide the amplitude distributions across frequencies, while the Fast Fourier Transform provides the computational efficiency to sum thousands of wave components in real time.

2.1.2 Tessendorf's Approach

The standard method for generating realistic ocean surfaces in computer graphics was formalized by Jerry Tessendorf in a series of SIGGRAPH course notes first presented in 1999 and revised through 2004 [59]. Although published as course notes rather than a peer-reviewed paper, Tessendorf's work is among the most cited references in ocean rendering and underpins virtually all modern CG water simulation systems. The approach had precursors in the oceanographic community, notably the work of Mastin, Watterberg, and Mareda, who first applied FFT to ocean rendering using the PM spectrum [37].

Tessendorf's method departs from the incompressible Navier-Stokes equations,

which, under the assumptions of non-rotational flow and conservative forces (gravity), reduce to Bernoulli's equation and Laplace's equation for the velocity potential. Linearizing the free-surface boundary conditions yields the surface wave equation, whose general solution is a superposition of plane waves governed by the dispersion relation (Equation 2).

The ocean surface height at horizontal position $\mathbf{x} = (x, z)$ and time t is expressed as

$$h(\mathbf{x}, t) = \sum_{\mathbf{k}} \tilde{h}(\mathbf{k}, t) \exp(i\mathbf{k} \cdot \mathbf{x}) \tag{4}$$

where the sum runs over a discrete grid of two-dimensional wave vectors $\mathbf{k} = (k_x, k_z)$ with $k_x = 2\pi n/L_x$ and $k_z = 2\pi m/L_z$, for integers n and m bounded by the grid resolution $N \times M$. This sum is computed efficiently via the inverse Fast Fourier Transform.

The complex spectral amplitudes $\tilde{h}(\mathbf{k}, t)$ are initialized from a statistical model. Tessendorf uses the Phillips spectrum [45] with a directional spreading factor

$$P_h(\mathbf{k}) = A \frac{\exp(-1/(kL)^2)}{k^4} \left| \hat{\mathbf{k}} \cdot \hat{\boldsymbol{\omega}} \right|^2 \tag{5}$$

where $L = V^2/g$ is a length scale determined by the wind speed V, and $\hat{\boldsymbol{\omega}}$ is the wind direction unit vector. Initial amplitudes are drawn as

$$\tilde{h}_0(\mathbf{k}) = \frac{1}{\sqrt{2}} (\xi_r + i\xi_i) \sqrt{P_h(\mathbf{k})} \tag{6}$$

where ξ_r and ξ_i are independent draws from a standard normal distribution. Time evolution uses the dispersion relation to rotate each spectral component

$$\tilde{h}(\mathbf{k}, t) = \tilde{h}_0(\mathbf{k}) e^{i\omega(k)t} + \tilde{h}_0^*(-\mathbf{k}) e^{-i\omega(k)t}. \tag{7}$$

This preserves the conjugation property $\tilde{h}^*(\mathbf{k}, t) = \tilde{h}(-\mathbf{k}, t)$, ensuring the resulting height field is real-valued.

The slope vector, needed for surface normal computation and optical modeling, is obtained analytically from additional FFTs

$$\boldsymbol{\epsilon}(\mathbf{x}, t) = \nabla h(\mathbf{x}, t) = \sum_{\mathbf{k}} i\mathbf{k}\, \tilde{h}(\mathbf{k}, t) \exp(i\mathbf{k} \cdot \mathbf{x}). \tag{8}$$

A key contribution of Tessendorf's work is the introduction of *choppy waves* through horizontal displacement. The basic FFT height field produces rounded wave profiles characteristic of calm conditions. To create the sharper crests observed in real ocean waves,

each surface point is displaced horizontally

$$\mathbf{D}(\mathbf{x}, t) = \sum_{\mathbf{k}} -i\frac{\mathbf{k}}{|\mathbf{k}|}\,\tilde{h}(\mathbf{k}, t)\exp(i\mathbf{k}\cdot\mathbf{x}) \tag{9}$$

and scaled by a choppiness parameter λ. This displacement concentrates surface material at wave crests, producing the characteristic sharp peaks and broad troughs of ocean waves. When the displacement is too large, the surface folds over itself — a non-physical artifact that nonetheless indicates where whitecaps and foam would form. The Jacobian of the displacement map

$$J(\mathbf{x}) = \left(1 + \lambda\frac{\partial D_x}{\partial x}\right)\left(1 + \lambda\frac{\partial D_y}{\partial y}\right) - \lambda^2\frac{\partial D_x}{\partial y}\frac{\partial D_y}{\partial x} \tag{10}$$

signals fold regions where $J < 0$, providing a physically motivated foam mask.

Subsequent work has refined the spectral input. Horvath [28] replaced the Phillips spectrum with the more physically accurate depth-dependent extension of the deep-water, based on Texel-Marsen-Arsloe (TMA) datasets and empirical directional spreading functions, improving the realism of coastal and shallow-water wave fields.

2.1.3 Blender Implementation

The open-source 3D modeling software Blender implements Tessendorf's algorithm through its Ocean modifier, making FFT-based ocean synthesis accessible for synthetic data generation without requiring custom code [3]. The implementation traces back to Drew Whitehouse's Houdini Ocean Toolkit (2005), which was ported to Blender by Hamed Zaghaghi and later modernized by Matt Ebb with OpenMP multi threading and OpenEXR baking support.

The Ocean modifier exposes the key parameters of Tessendorf's method as user-controllable settings. The *Resolution* parameter sets the FFT grid size as a power of 2 (typically 128 to 1024), determining the number of wave components summed. *Spatial Size* defines the physical extent of the ocean patch in meters. *Wind Speed* controls the wave size through the relationship $L = V^2/g$, and *Wave Scale* provides an additional amplitude multiplier. The *Choppiness* parameter corresponds directly to the displacement scale λ in Equation 9, while *Smallest Wave* acts as a low-pass filter suppressing high-frequency components. The *Depth* parameter modifies the dispersion relation for finite-depth effects (Equation 2), and directional controls (*Wind Alignment*, *Direction*, *Damping*) shape the angular distribution of wave energy. Since Blender 3.x, multiple spectral models are available: Phillips, Pierson-Moskowitz, JONSWAP, and TMA. The modifier outputs displacement maps, normal maps, and Jacobian-based foam maps as

vertex attributes, all of which can be baked to OpenEXR image sequences for offline rendering.

For synthetic dataset generation, Blender's Cycles path-tracing renderer produces physically based images when combined with appropriate material shaders (Fresnel reflectance, volumetric absorption, caustics). The Python scripting API enables full automation of the rendering pipeline: camera placement at controlled altitudes, randomization of ocean parameters (wind speed, direction, choppiness, seed), lighting conditions (sun angle, sky model), and batch rendering. This programmatic control is essential for generating large-scale labeled datasets where the altitude label is derived directly from the camera position.

The use of 3D rendering engines for synthetic training data has been validated across computer vision domains. Denninger et al. [16] developed BlenderProc, a procedural pipeline for photorealistic synthetic data generation using Blender's rendering capabilities. Tobin et al. [60] introduced *domain randomization*, the principle that randomizing rendering parameters sufficiently makes real-world images appear as just another variation of the synthetic domain. It demonstrated successful sim-to-real transfer using only synthetic RGB data. Tremblay et al. [62] further showed that fine-tuning on real data after synthetic pretraining with domain randomization outperforms training on real data alone, establishing the mixed-training curriculum adopted in this work.

2.2 Quadcopter Movement Based Distortion

Images captured by cameras mounted on quadcopters are subject to motion blur caused by the relative movement between the camera and the scene during the exposure interval. For a downward-facing camera imaging a water surface, two primary motion components contribute to image degradation: translational movement in the horizontal plane and rotational movement around the vertical (yaw) axis. Other rotational movements are not covered in this work, although further investigation is necessary for a more robust modeling. Understanding and modeling these degradation mechanisms is important both for training comprehensive models and for designing appropriate data augmentation strategies.

2.2.1 Linear Blurring

Translational motion during the camera exposure interval produces a spatially invariant blur characterized by a linear Point Spread Function (PSF). When the camera undergoes uniform linear displacement during exposure, the observed image $g(x, y)$ is

related to the sharp image $f(x, y)$ by the convolution

$$g(x, y) = \int_0^T f(x - v_x t,\ y - v_y t)\, dt + n(x, y) \tag{11}$$

where (v_x, v_y) is the velocity of the camera projected onto the image plane, T is the exposure time, and $n(x, y)$ represents additive sensor noise [22]. For motion along the x-axis with total displacement $L = v_x T$ pixels, the PSF reduces to

$$h(x, y) = \frac{1}{L} \operatorname{rect}\left(\frac{x}{L}\right) \delta(y), \tag{12}$$

a uniform line segment of length L. In the frequency domain, this corresponds to the transfer function

$$H(u, v) = \operatorname{sinc}(uL) \cdot e^{-j\pi uL}. \tag{13}$$

The sinc envelope introduces periodic zeros in the frequency domain, causing irreversible loss of spatial frequency information at specific frequencies. The linear phase term reflects the positional shift of the blurred image centroid.

Potmesil and Chakravarty [46] provided the foundational treatment of motion blur in computer-generated images, modeling the PSF as a generalized system-transfer function derived from the object path and camera exposure time. For UAV imagery specifically, Sieberth et al. [55] demonstrated that blur from forward flight, vibrations, wind gusts, and turbulence is pervasive in UAV datasets, and proposed automatic detection methods based on frequency-domain analysis. The s photogrammetric measurements from UAV imagery [54].

For altitude estimation, translational motion blur attenuates the high-frequency texture content of water surface images, which is precisely the feature domain that encodes altitude information. As demonstrated by Pei et al. [42], motion blur significantly degrades CNN classification performance, and degradation removal only partially restores it. Hendrycks and Dietterich [26] quantified this effect systematically by benchmarking neural network robustness to motion blur at multiple severity levels, establishing that blur is among the most damaging common corruptions for CNN accuracy. These findings motivate the inclusion of motion blur as a data augmentation strategy during training, so that the model encounters blurred samples and learns to be robust to this degradation.

2.2.2 Circular Blurring

Rotation of the quadcopter around its yaw axis during exposure produces a qualitatively different blur pattern. Unlike translational blur, yaw-induced blur is *spatially variant*: the blur arc length at each pixel depends on its distance from the rotation center.

For a pure yaw rotation of angle θ during exposure, a pixel at distance r from the optical center traces an arc of length $r\theta$, producing a tangential blur whose magnitude increases radially [64].

The spatially variant PSF for rotational blur can be expressed as

$$h_r(x, y; x_c, y_c) = \frac{1}{\theta}\,\delta\Big(\sqrt{(x - x_c)^2 + (y - y_c)^2} - r\Big) \tag{14}$$

for $|\angle(x - x_c, y - y_c) - \angle_0| \leq \theta/2$, where (x_c, y_c) is the rotation center and r is the radial distance. Unlike the linear case, this PSF cannot be expressed as a single global convolution kernel, making modeling substantially more complex.

For quadcopter applications, yaw-induced blur is particularly relevant during turning maneuvers and in the presence of wind-induced yaw disturbances. The circular blur pattern produces a characteristic radial smearing in the frequency domain, attenuating angular frequency components rather than the linear zeros produced by translational blur. In the context of synthetic data augmentation, rotational blur can be simulated by applying a spatially variant circular convolution kernel parameterized by the yaw rate and exposure time.

2.3 Feature Engineering and Pattern Recognition

The altitude of a camera above a water surface encodes itself in the captured image through several measurable visual properties: the scale and visibility of surface texture, the spatial distribution and intensity of specular reflections, and the spectral energy distribution across spatial frequencies. This subsection describes three complementary feature extraction techniques that exploit these physical relationships.

2.3.1 Channel Isolation (HSV)

The appearance of a water surface varies considerably with environmental conditions: turbidity, dissolved organic matter, algae content, and bottom reflectance all affect the color of the water. However, these chromaticity variations are uncorrelated with altitude and constitute confounding factors for an altitude estimation model [50, 43]. The altitude-dependent features are primarily encoded in brightness and texture patterns, the scale of visible wave structures, the intensity and distribution of specular reflections, and the overall contrast of the surface.

The HSV (Hue, Saturation, Value) color space decouples brightness from chromaticity, making the Value channel a natural choice for isolating altitude-relevant features

while suppressing color-based confounds. The Value component is defined as

$$V = \max(R, G, B). \tag{15}$$

This definition preserves the peak brightness of each pixel regardless of its hue, which is particularly important for water surfaces where specular reflections create localized intensity maxima that correlate with surface geometry and, by extension, with the viewing altitude.

The empirical justification for HSV isolation in water surface analysis has been established across multiple studies. Pekel et al. [43] demonstrated the superiority of HSV over RGB for automated water surface detection at continental scale, finding that the separability of water pixels from non-water pixels is significantly higher in HSV space. Samaranayake et al. [51] explicitly showed altitude-dependent visual transitions in their Drone-Water dataset: at low altitudes (1–2 m), individual ripples are visible and optical flow from surface texture serves as the primary detection cue; at medium altitudes (10–30 m), wave patterns average out and the uniform texture itself becomes the discriminative feature. Choi et al. [8] confirmed through attention map analysis that CNNs estimating water surface elevation focus on regions of significant intensity change due to wave propagation, validating that brightness-based features drive the estimation.

A critical implementation detail is that the HSV Value channel ($V = \max(R, G, B)$) differs from standard grayscale conversion, which applies the ITU-R BT.601 luminance formula ($Y = 0.299R{+}0.587G{+}0.114B$). The grayscale formula is a weighted average that can attenuate the very intensity peaks that encode altitude information, while the max operation in the Value channel preserves them. This distinction has direct implications for the preprocessing pipeline.

2.3.2 Image FFT

The two-dimensional Fast Fourier Transform decomposes an image into its spatial frequency components, revealing periodic structures and texture properties that are not readily apparent in the spatial domain. The 2D DFT of an $M \times N$ image $f(x, y)$ is defined as

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y)\, e^{-j2\pi(ux/M + vy/N)}. \tag{16}$$

The magnitude spectrum $|F(u, v)|$ encodes the energy distribution across spatial frequencies, while the phase spectrum $\angle F(u, v)$ encodes the spatial arrangement of those frequencies. The FFT algorithm [10] computes this transform in $O(N \log N)$ operations, making frequency-domain analysis practical for real-time applications.

The relationship between spatial frequency content and texture scale is well established: coarse textures concentrate energy at low spatial frequencies, while fine textures spread energy to higher frequencies [23]. For water surfaces viewed from varying altitudes, this relationship is physically grounded. At low altitudes, the camera resolves individual capillary waves and ripples, producing high-frequency texture content. As altitude increases, each pixel integrates over a larger ground area, effectively low-pass filtering the surface: small-scale wave structures are no longer resolvable, and the image becomes dominated by large gravity waves or appears nearly featureless. This altitude-dependent frequency shift provides a robust feature for regression.

Useful frequency-domain features for water surface altitude estimation include the radial energy distribution (total energy as a function of distance from the DC component), the spectral centroid and bandwidth (measures of the dominant texture period and its spread), and the ratio of high-frequency to low-frequency energy. Xiong et al. [69] demonstrated rotation-invariant texture features extracted from circular regions of 2D FFT magnitude spectra. Yang et al. [71] established empirical relationships between wind speed, surface roughness, and texture features extracted from visible-band sea surface images, showing that frequency-domain texture measures correlate strongly with physical surface properties.

In the context of a multi-input model, FFT-derived features serve as a complementary input branch alongside the raw image, providing the network with explicit frequency-domain information that a convolutional architecture may otherwise need many layers to discover implicitly.

2.3.3 Shi and Tomasi's Good Features To Track

The detection of salient point features on water surfaces provides information about the density and distribution of specular reflections, which correlate inversely with altitude. The Shi-Tomasi corner detector [52], building on the earlier work of Harris and Stephens [24] and the Kanade-Lucas-Tomasi tracking framework [61], identifies image points that are well-suited for tracking across frames.

The algorithm computes the *structure tensor* (also called the autocorrelation matrix or second-moment matrix) over a local Gaussian-weighted window $w(x, y)$

$$\mathbf{M} = \sum_{x,y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \tag{17}$$

where I_x and I_y are the image gradients computed via convolution with derivative filters (typically Sobel operators). The eigenvalues λ_1 and λ_2 of $\mathbf{M}$ characterize the local

intensity structure: both eigenvalues large indicates a corner (strong gradients in two directions), one large and one small indicates an edge, and both small indicates a flat region.

The Shi-Tomasi criterion declares a point a "good feature" if

$$\min(\lambda_1, \lambda_2) > \tau \tag{18}$$

where τ is a quality threshold. This contrasts with the Harris detector, which uses the response function $R = \det(\mathbf{M}) - k \cdot \text{trace}(\mathbf{M})^2 = \lambda_1\lambda_2 - k(\lambda_1 + \lambda_2)^2$, where $k \approx 0.04$–0.06 is an empirical constant [24]. The Shi-Tomasi criterion produces a cleaner geometric decision boundary in eigenvalue space and is more directly interpretable as a constraint on the minimum gradient strength in any direction.

For water surface analysis, specular reflection peaks (sun glint hotspots) create localized high-gradient regions that are detected as corners in the Shi-Tomasi sense, as seen in 1. At low altitudes, many micro-facets of the undulating surface can individually produce detectable reflections, resulting in a high density of detected features. As altitude increases, only large-scale wave structures produce detectable reflections, and the feature count decreases. The density and spatial distribution of Shi-Tomasi features thus serve as altitude-correlated handcrafted features that complement learned CNN representations.

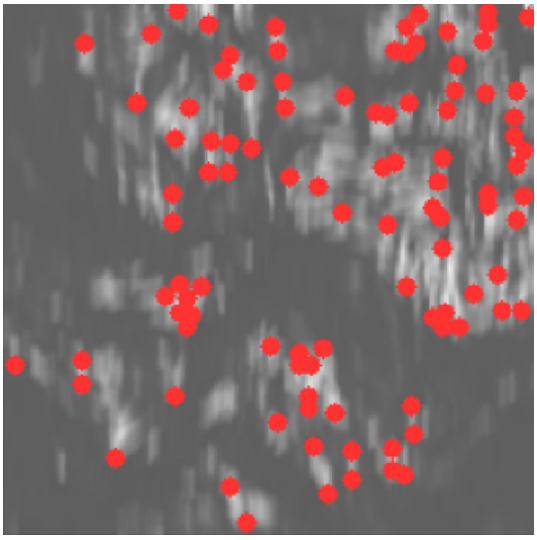

Figure 1: Example of points detected in a sample, using in Shi-Tomasi's algorithm.

2.4 Machine Learning

This subsection describes the learning algorithms used for altitude regression: convolutional neural networks for image-based feature learning, and gradient boosting for

tabular feature regression. It also covers the multi-input architecture that fuses both modalities.

2.4.1 Convolutional Neural Network

Convolutional Neural Networks (CNNs) learn hierarchical representations of visual data through sequences of trainable convolutional filters, nonlinear activations, and spatial pooling operations. Each convolutional layer applies a set of learned kernels to the input feature maps, producing output feature maps that capture increasingly abstract patterns: edges and textures in early layers, parts and shapes in intermediate layers, and object-level semantics in deeper layers [39]. A visual representation of a neural network is shown in Fig.2.

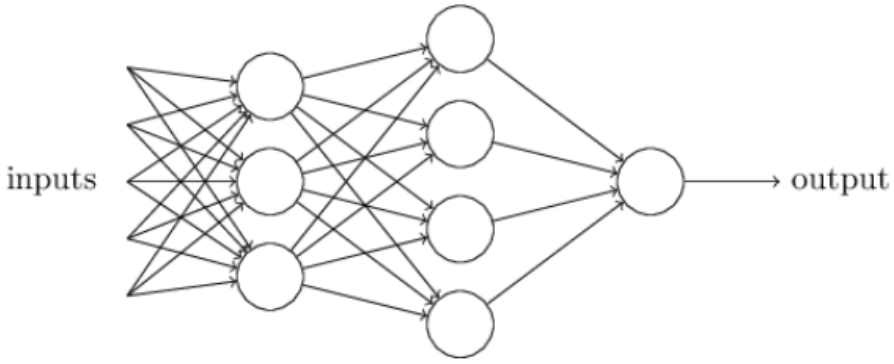

Figure 2: Example of three layers CNN. [39]

For regression tasks such as altitude estimation, the classification head of a standard CNN is replaced with a regression head: the final convolutional feature maps are reduced to a fixed-length vector (typically via global average pooling), passed through one or more fully connected layers, and mapped to a single scalar output through a linear activation. The model is trained by minimizing a loss function that measures the discrepancy between predicted and ground-truth altitude values, commonly the Huber loss, which combines the smooth gradients of mean squared error near zero with the robustness to outliers of mean absolute error.

2.4.1.1 VGG: The VGG architecture, introduced by Simonyan and Zisserman [56], demonstrated that network depth using exclusively small (3×3) convolution filters yields superior performance compared to shallower networks with larger kernels. The key insight is that a stack of two 3×3 convolutional layers has the same effective receptive field as a single 5×5 layer, but introduces two nonlinearities and uses fewer parameters ($2 \times 9C^2 = 18C^2$ versus $25C^2$ for C channels). Three stacked 3×3 layers match a 7×7 receptive field with even greater parameter savings ($27C^2$ versus $49C^2$).

Consisting of 13 convolutional layers, the VGG-16 variant is organized in five blocks with a consistent filter progression of $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$ channels, separated by 2×2 max-pooling, followed by three fully connected layers ($4096 \rightarrow 4096 \rightarrow 1000$), as shown in 3, totaling approximately 138 million parameters. For transfer learning to altitude regression, the final classification layer is replaced with a single linear output neuron, and the pretrained convolutional blocks serve as a feature extractor. The depth and regularity of VGG's architecture make it a strong baseline for regression tasks where the input images share low-level statistical properties with natural images, and is used as a base model for the present work.

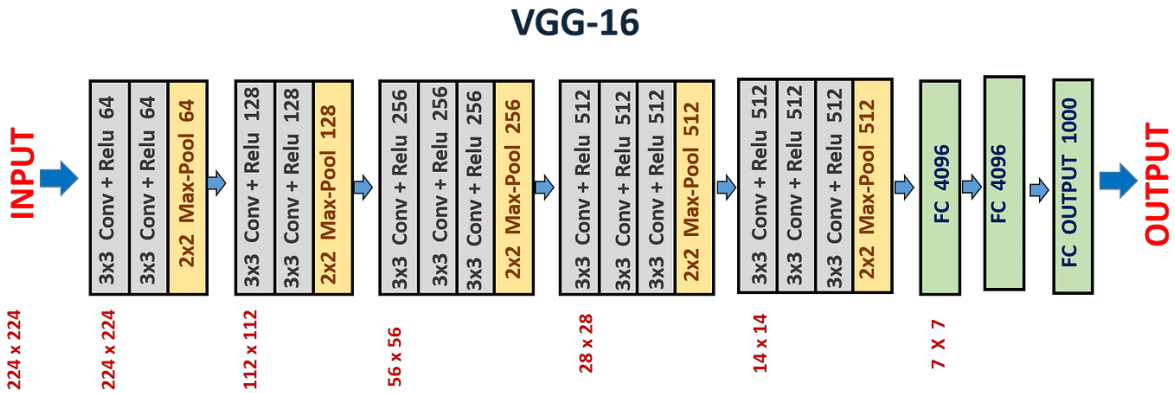

Figure 3: Flattened architecture of VGG16. From [34]

2.4.1.2 Multi-input CNN: When both image data and tabular features (such as FFT statistics, gradient magnitudes, and feature counts from Shi-Tomasi detection) are available, a multi-input architecture can combine both modalities through *feature fusion*. Three general fusion strategies exist [20, 74]:

Early fusion concatenates modalities at the input level, requiring compatible spatial formats, but preserving rich cross-modal information from the first layer onward.

Late fusion processes each modality through independent streams a CNN branch for images and a multilayer perceptron (MLP) branch for tabular features, and merges their high-level representations at the fully connected layers via concatenation. This is the simplest and most commonly used approach.

Intermediate fusion merges at hidden layers, allowing the network to learn joint representations while preserving modality-specific structure in early layers. Wolf et al. [66] proposed Dynamic Affine Feature Map Transform (DAFT), where an auxiliary MLP generates scale and offset parameters from tabular data to condition CNN feature maps an elegant mechanism that modulates visual processing based on numerical context.

Joze et al. [31] introduced the Multimodal Transfer Module (MMTM), which uses squeeze-and-excitation operations across modality-specific CNN streams for intermediate fusion, demonstrating state-of-the-art results on multimodal recognition tasks.

For altitude estimation, the late fusion approach is adopted: a CNN backbone (VGG based) processes the HSV Value channel image, while a parallel MLP processes a vector of handcrafted features (FFT spectral statistics, Shi-Tomasi feature count, gradient statistics). The two streams are concatenated at the fully connected layers, and the fused representation is mapped to a single altitude prediction.

2.4.2 LightGBM

LightGBM (Light Gradient Boosting Machine) is a gradient boosting framework that builds an ensemble of decision trees sequentially, where each new tree corrects the residual errors of the existing ensemble [33]. It introduces two key algorithmic innovations that distinguish it from prior methods such as XGBoost [9].

The first innovation is *Gradient-based One-Side Sampling* (GOSS), which retains all training instances with large gradients (poorly predicted samples that contribute most to learning) while randomly subsampling from instances with small gradients. This exploits the insight that well-predicted data points contribute little to information gain during tree splitting, enabling significant speedups without meaningful accuracy loss.

The second innovation is *Exclusive Feature Bundling* (EFB), which identifies and bundles mutually exclusive features (features that are rarely simultaneously nonzero) into single composite features. This reduces the effective dimensionality of the feature space, further accelerating the split-finding procedure.

Combined with histogram-based splitting (discretizing continuous features into fixed bins) and leaf-wise tree growth (splitting the leaf with maximum loss reduction rather than growing level-by-level), LightGBM achieves training speedups of up to 20× compared to XGBoost with comparable accuracy on standard benchmarks.

In the context of this work, LightGBM serves as a complementary model to the CNN, operating on the vector of handcrafted features extracted from the water surface images (FFT spectral statistics, gradient magnitudes, Shi-Tomasi feature counts, pixel intensity statistics). This hybrid approach is supported by the finding of Shwartz-Ziv and Armon [53], who systematically showed that gradient boosting methods outperform deep learning on tabular data, and that ensembles combining deep models with gradient boosting achieve the best overall performance. Breiman's Random Forests [4] provide a natural comparison baseline within the ensemble learning family.

2.5 Metrics

The evaluation of an altitude regression model requires a set of complementary metrics that capture different aspects of prediction quality: absolute error magnitude, scale-dependent versus scale-independent accuracy, and the ability to explain model decisions.

2.5.1 Accuracy, Precision and Error

The most fundamental distinction in regression evaluation is between accuracy (closeness to the true value) and precision (consistency of repeated measurements). A model can be precise but inaccurate if it exhibits systematic bias (consistent over- or under-estimation), or accurate on average but imprecise if individual predictions scatter widely around the true value. In the context of altitude estimation, both properties are critical: systematic bias affects altitude control stability, while high variance reduces the reliability of individual measurements.

The choice between error metrics depends on the assumed error distribution. Hodson [27] established that Root Mean Squared Error (RMSE) is the optimal metric when errors follow a Gaussian distribution, as it corresponds to the maximum likelihood estimator of the standard deviation:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2} \tag{19}$$

Mean Absolute Error (MAE) is optimal for Laplacian-distributed (heavy-tailed) errors:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i| \tag{20}$$

The gap between RMSE and MAE serves as a diagnostic: a large gap indicates the presence of outliers (heavy-tailed errors), since RMSE penalizes large errors quadratically while MAE penalizes them linearly. For altitude estimation where safety is paramount, RMSE's quadratic outlier penalty is desirable because large errors are disproportionately dangerous. Reporting both metrics provides complementary information about average and worst-case performance.

2.5.2 Regression Evaluation

Beyond absolute error, a comprehensive evaluation of altitude regression requires scale-independent metrics, goodness-of-fit measures, and distribution-characterizing statis-

tics. The coefficient of determination quantifies the proportion of variance in the target variable explained by the model:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{21}$$

where $\bar{y}$ is the mean of the ground-truth values. An R^2 value close to 1 indicates that the model captures nearly all altitude variation, while $R^2 \leq 0$ indicates performance no better than predicting the mean.

Mean Absolute Percentage Error (MAPE) provides a scale-independent measure of accuracy:

$$\text{MAPE} = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{\hat{y}_i - y_i}{y_i}\right| \tag{22}$$

MAPE is particularly useful for altitude estimation because it normalizes the error by the true altitude: an error of 10 cm at 50 cm altitude (20%) is more operationally significant than the same absolute error at 500 cm (2%).

The monocular depth estimation community follows the Eigen evaluation protocol [18], reporting threshold accuracy metrics ($\delta < 1.25$, 1.25^2, 1.25^3) alongside AbsRel and SqRel. However, UAV altitude estimation is a single-value regression task rather than per-pixel depth prediction, requiring a different metric emphasis. The Rajapaksha et al. survey [48], covering over 160 depth estimation papers, confirms that MAE, RMSE, R^2, and MAPE with cross-validated confidence intervals constitute the accepted evaluation standard for altitude regression benchmarks in IEEE, ACM, and Springer publications.

For stratified evaluation, computing these metrics within altitude bins (e.g., 50–100 cm, 100–200 cm, 200–400 cm, 400–800 cm) reveals range-dependent performance patterns that global metrics can mask. The median absolute error (MedAE) provides an outlier-robust measure of central tendency, the 95th percentile of absolute error characterizes worst-case performance, and the standard deviation of errors quantifies prediction consistency. For small datasets, k-fold cross-validation (typically 5 or 10 folds) with stratified altitude bins is essential, reporting mean $\pm$ standard deviation across folds.

3 RELATED WORKS

The problem addressed in this work, monocular altitude estimation above water surfaces, occupies a notably underexplored niche in the literature. While substantial research exists on monocular depth estimation over rigid, textured surfaces [12] [30], and on CNN-based UAV perception for obstacle avoidance [6] [32], autonomous navigation [35], and object detection and tracking [67] [38], no prior work has specifically targeted the regression of absolute altitude from a single downward-facing image of a water surface. This gap is significant: water surfaces defeat conventional altimetry sensors (LIDAR, ultrasonic, barometric) due to specular reflectivity, surface dynamics, and evaporation-driven barometric drift, yet they remain a critical operational environment for UAVs in offshore inspection, maritime search-and-rescue, and environmental monitoring. The contribution of the present work is therefore exclusive in its formulation, combining HSV-based feature extraction, photorealistic synthetic data generation, and CNN-based regression to address a problem for which no existing solution is adequate.

To contextualize this contribution, the following subsections review the closest related efforts in three areas: CNN-based UAV estimation systems, water surface detection and proximity sensing, and training with synthetically generated datasets.

3.1 CNN-Based UAV Estimations

Altitude estimation for unmanned aerial vehicles has traditionally relied on processing data from barometers, ultrasonic sensors, and LiDAR through classical filtering algorithms [12]. Each technology carries distinct limitations: barometric altimeters suffer from drift due to atmospheric pressure variations; ultrasonic rangefinders fail over surfaces that absorb or scatter acoustic energy; and LiDAR performance degrades severely over specular or semi-transparent surfaces [65]. Even modern sensor fusion approaches based on Kalman and complementary filters cannot compensate for scenarios in which all constituent sensors produce unreliable readings simultaneously, as occurs over highly reflective surfaces such as water, snow, and sand dunes [21].

The application of CNNs to UAV perception tasks has demonstrated that vision-based approaches can complement or replace traditional sensors in constrained scenarios. A. A. Cabrera-Ponce and J. Martinez-Carranza proposed a seven-layer Single Shot Detection CNN (SSD7) for onboard identification of landing conditions, deployed within a ROS framework on a low-power Intel Stick M3 and achieving 13 fps in real time [5]. KOURIS, Alexandros and BOUGANIS, Christos-Savvas developed a self-supervised system for indoor UAV navigation, where an AlexNet-based architecture processed RGB images fused with infrared and ultrasonic data in a parallel-input configuration [35]. The system out-

puts a set of navigation options with associated direction and speed for collision avoidance and autonomous flight. HUA et al. proposed a lightweight object tracking model with distance calculation for real-time UAV operation on mobile devices [29]. The architecture was based on MobileNet variants (MobileNetV1, MobileNetV2, and E-MobileNet) with seven hidden layers, achieving 56 fps on an NVIDIA Jetson AGX Xavier with an average accuracy of 87.2More recently, ZHANG, Xupei et al. presented VIAE-Net, an end-to-end altitude estimation system that fuses monocular visual features with inertial measurements for UAV autonomous landing, demonstrating the viability of deep learning for metric altitude recovery [73].

These works collectively establish that CNNs are capable of extracting spatial and metric information from monocular imagery in UAV contexts. However, none of them addresses the specific challenge of altitude estimation over water, where the absence of stable visual landmarks, the variability of surface texture with wave conditions, and the confounding effects of specular reflection demand a fundamentally different approach.

3.2 Water Surface Detection and Proximity Sensing

A small but growing body of work has addressed the problem of detecting and characterizing water surfaces from aerial and ground-level imagery, but none have formulated the problem as altitude regression.

SAMARANAYAKE et al. introduced two techniques for water surface identification from UAV-mounted cameras [51]. The first, UNet-RAU, is a segmentation architecture based on UNet with Reflection Attention Units that uses the specular reflection properties of water to segment water pixels from front-facing camera views, achieving an F1-score of 80.97% on a custom Drone-Water-Front dataset. The second method, Dense Optical Flow based Water Detection (DOF-WD), identifies water surfaces in downward-facing video streams by leveraging rotor downwash-generated ripples at low altitudes and natural texture features at higher altitudes. While this work demonstrates that water surfaces can be reliably detected from UAV imagery, including from downward-facing cameras, the task remains binary detection (water vs. non-water) rather than metric altitude estimation. The present work extends beyond this boundary by treating the water surface not merely as a class to be detected, but as a visual signal from which continuous altitude values can be regressed.

Chang and Xu proposed the DCU-Net, a densely connected U-Net architecture combining DenseNet-121 and U-Net for water puddle segmentation on road surfaces [7]. The model was designed for deployment in IoT-enabled environments and demonstrated improved detection accuracy compared to standard segmentation baselines. Although the application domain shares the geometric configuration of a camera perpendicular to

a water surface, the objective is semantic segmentation rather than depth or altitude regression. Nevertheless, the work confirms that CNN architectures can learn meaningful features from water surface imagery captured in a top-down perspective, which is a relevant finding for the present research.

Zhengyang Lu and Ying Chen addressed self-supervised monocular depth estimation of water scenes by implementing a specular reflection prior, acknowledging that standard depth estimation methods fail when confronted with the reflective properties of water [36]. Their work confirms the central premise of the present thesis: that water surfaces require specialized treatment in vision-based depth and altitude estimation systems.

Jiaqi Chen and Haijiang Liu applied CNNs to estimate water surface elevation from laboratory imagery, evaluating the baseline, VGGNet, and ResNet architectures [8]. Their results showed that CNN performance is sensitive to the choice of network structure but relatively insensitive to network depth, and that the learned attention hot spots correspond to regions of significant intensity change caused by wave propagation. This finding supports the hypothesis of the present work that altitude-related cues are encoded in the texture and brightness patterns of the water surface.

3.3 Training with Synthetically Generated Datasets

The use of synthetic data for training deep learning models is a well-established practice in domains where real image banks are sparse, expensive to produce, or dangerous to collect [68] [47] [49] [44] [1]. This strategy enables safer development cycles, particularly in proof-of-concept stages, by eliminating the need for costly field equipment and prolonged human exposure to hazardous environments. In the context of the present work, no open dataset exists containing downward-facing images of water surfaces with associated altitude labels, making photorealistic rendering in Blender not merely convenient but a methodological necessity.

Several related works illustrate the effectiveness of synthetic data pipelines across diverse domains. [44] addressed the problem of motion blur in UAV-captured images of wind turbine blades, for which adequate cataloged images are unavailable. They combined three strategies: compositing images of moving and stationary propellers, generating hybrid motion-blurred images through a generative network, and simulating propeller motion to produce virtually blurred images. The resulting bank was processed by M-DeblurGANv2, a network based on the DeblurGAN architecture, as illustrated in Figure 4.

ZHANG et al. used a Faster R-CNN network to detect forest fire smoke, over-

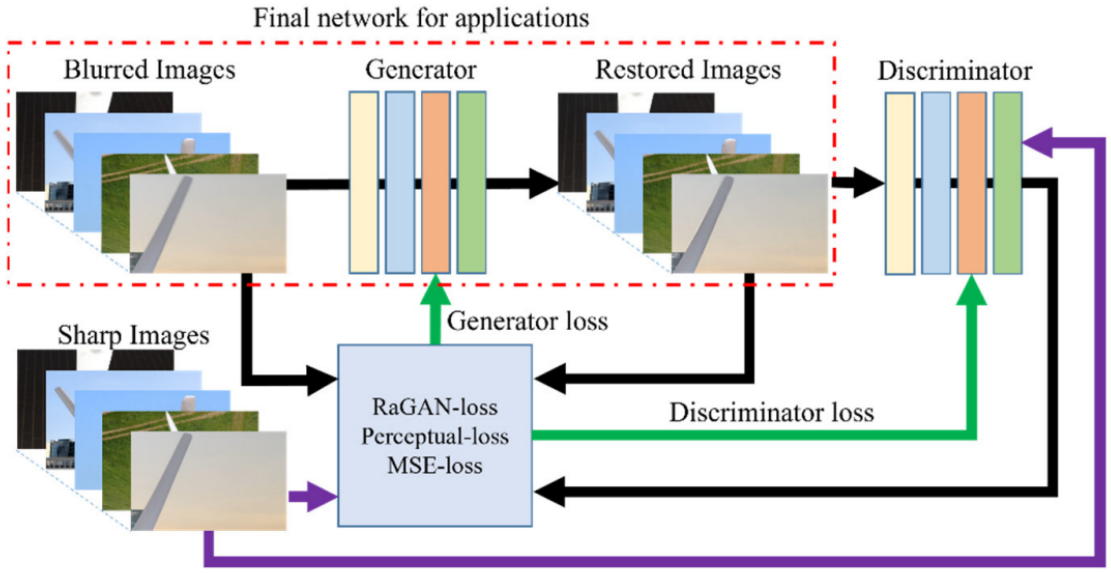

Figure 4: Flowchart of DeblurGANv2-based motion deblur of wind turbine blade images. [44]

coming the sparsity of suitable smoke images by generating two synthetic datasets: one with real smoke filmed against a green screen and composited onto landscape footage, and another with smoke rendered entirely in Blender [68]. The model trained on Blender-rendered smoke outperformed the one trained on composited real smoke, demonstrating that fully synthetic data can achieve superior results when the rendering pipeline adequately captures the visual characteristics of the target phenomenon.

RAJPURA et al. tackled the scarcity of labeled image data for food package detection in refrigerators [49]. The authors produced a fully synthetic bank of 4,000 Blender-rendered images and a hybrid bank combining 3,600 synthetic images with 400 real images. Training a GoogleNet Fully Convolutional network for 200 epochs, the hybrid dataset outperformed both the synthetic-only and real-only configurations, reinforcing the value of mixing synthetic and real data.

ACHARYA et al. created three-dimensional environments in Blender to generate image samples at distinct poses for mobile device pose estimation [1]. A PoseNet network based on GoogLeNet with 23 layers and six inception modules was trained on these synthetic images, achieving localization accuracy of approximately 2 meters in real time.

The synthetic data strategy adopted in the present work follows the same principles, with the critical difference that the target domain of water surface, presents unique rendering challenges related to physically-based wave modeling, subsurface scattering, and view-dependent specular behavior, which were addressed through Blender's ocean modifier and physically-based shading pipeline. The domain randomization principles established by [60] and [62] further support this approach, showing that variability in

synthetic rendering parameters can improve generalization to real-world conditions.

3.4 Broader Applications and Future Directions

The altitude estimation capability developed in this work has potential implications beyond the immediate application of single-UAV flight control. Xu et al. introduced a comprehensive micro-UAV swarm system for indoor perception in partially known environments, proposing integrated solutions for communication routing, multisensor localization, and cooperative decision-making [70]. Swarm systems of this nature demand that each individual UAV maintain reliable altitude estimation with minimal computational overhead, as the processing budget must be shared across navigation, communication, and coordination tasks. A lightweight, vision-based altitude estimator such as the one proposed here could serve as a complementary sensor for swarm members operating over water, where traditional altimetry is unreliable and the weight and power constraints of micro-UAVs preclude the addition of dedicated ranging hardware.

The convergence of these research directions, regard CNN-based metric estimation, water surface perception, synthetic data generation, and swarm UAV architectures, defines the space in which the present work makes its contribution: a problem that is simultaneously well-motivated by practical needs and inadequately addressed by existing methods.

4 MATERIALS AND METHODS

4.1 Artificial Dataset Synthesis

The strategy of generating training data synthetically, or augmenting real imagery with synthetic elements, is well established in computer vision research where labeled data are scarce or impractical to collect at scale. This approach enables fine-grained control over scene parameters, including geometry, illumination, and camera pose. It has been successfully applied to tasks ranging from smoke detection to fiducial marker localization [68, 47, 49]. Fully photorealistic scenes can be rendered with configurable lighting direction, color temperature, and intensity, and may include shadows and other environmental effects, producing synthetic datasets whose utility has been demonstrated in related works [44, 1].

Following this paradigm, a synthetic image bank was generated in Blender using a photorealistic water surface model that simulates quiescent fluid behavior representative of enclosed water bodies such as reservoirs, lakes, and large dams. Representative RGB samples are shown in Fig. 5. The wave geometry was synthesized using a three-dimensional time-varying noise algorithm to model surface undulation [19, 72]. Solar illumination was simulated with Blender's built-in sun lamp, which models a directional light source with spectrally configurable color and intensity, thereby capturing the chromatic variation in sunlight across different times of day and atmospheric conditions.

Images were rendered at 21 discrete altitude levels spanning 50 to 800 cm, specifically 50, 60, 70, 80, 90, 100, 120, 140, 160, 180, 200, 240, 280, 320, 360, 400, 480, 560, 640, 720, and 800 cm, with 9,000 raw images per level, yielding a total of 189,000 rendered images before preprocessing.

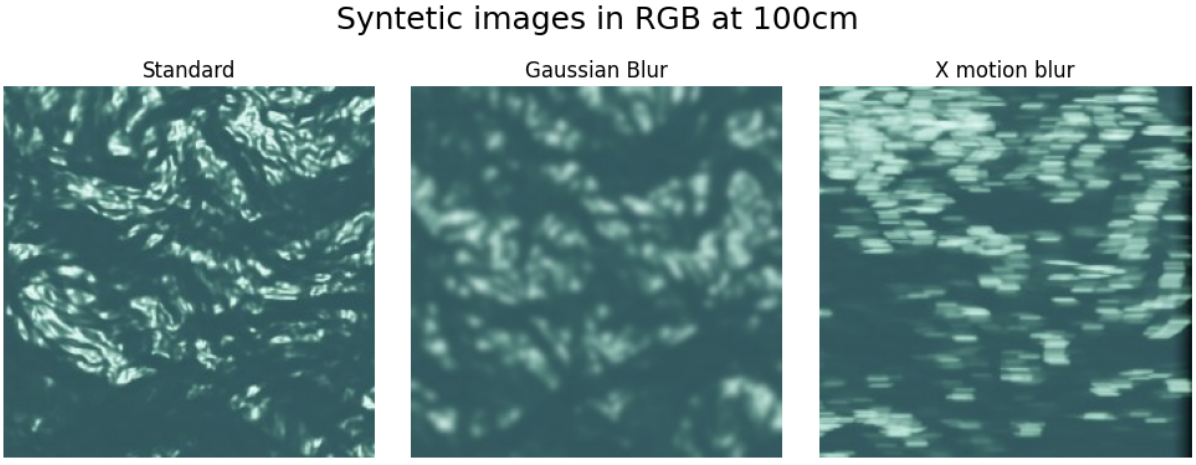

Figure 5: Synthetic RGB images rendered at representative altitude level of 100cm. Labels describe each sample distortion configuration.

4.1.1 Image Preprocessing Pipeline

Following rendering, a three-step preprocessing pipeline was applied offline to each image using a C++ routine built on OpenCV, as diagrammed in Fig. 6.

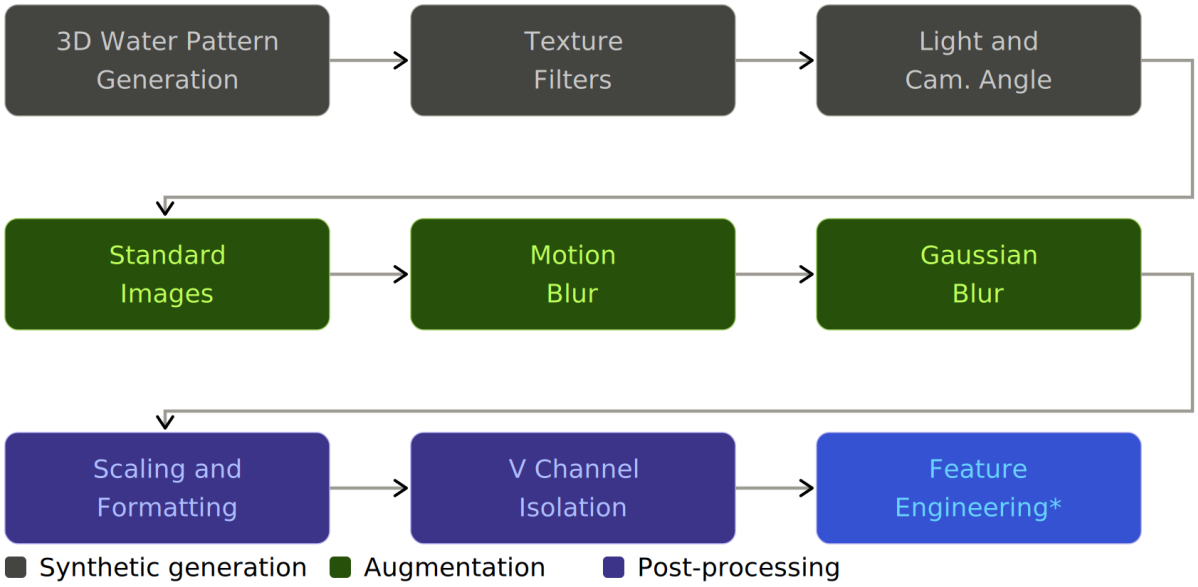

Figure 6: Image pipeline flowchart. For real-world images, only the Feature Engineering part is done. The Feature Engineering refers to the extraction, and is only used in late fusion models

1. **Motion blur augmentation.** Blur was applied in three directions: single-axis (simulating forward flight), diagonal (simulating oblique movement), and rotational (simulating yaw movement). This captures the range of apparent image motion induced by a quadrotor in operation [41, 13, 63].

2. **Gaussian blur augmentation.** A Gaussian kernel was applied to emulate optical blurring and condensation artifacts common on camera lenses in humid environments.

3. **HSV Value channel extraction.** Each RGB image was converted to the HSV color space and only the V channel was retained. As defined by $V = \max(R, G, B)$, the Value channel preserves specular highlights, which correlate with surface texture and thus encode altitude-dependent information, while discarding hue and saturation. This step reduces the network's sensitivity to color variations arising from biome-specific water coloration, suspended sediment, and algal content [50], and increases robustness to chromatic differences between the synthetic and real domains. Fig. 7 illustrates the contrast between the RGB input and the extracted V channel at three altitude levels.

For steps 1 and 2, kernel parameters were calibrated by fitting a Gaussian distribution to color statistics extracted from real-world images of a coastal site, ensuring that the simulated distortions are representative of field conditions.

This offline approach was preferred over runtime data augmentation in Keras for two reasons: it allows a precisely controlled and documented distribution of distortion types across the training set, and it avoids the cold posterior effect associated with insufficient augmentation diversity in large-scale training [40]. The resulting distribution of distortion configurations per altitude set is summarized in Table 1. Examples of preprocessed images without any blur are shown in Fig. 8, and examples with varying levels of motion blur are shown in Fig. 9.

Table 1: Preprocessing distribution for each altitude set of synthetically generated images. Each cell indicates the number of images per distortion configuration.

Distortion class (all 21 altitude sets)	No Gaussian blur	Gaussian blur
No motion blur	5,000	5,000
Single-axis motion blur	5,000	5,000
Diagonal motion blur	5,000	5,000

4.2 Computational Platform and Training Infrastructure

Blender rendering of the synthetic dataset was carried out on a workstation equipped with an AMD Ryzen™ 1600 six-core processor overclocked to 3.8 GHz, an NVIDIA GeForce GTX 970 GPU (4 GB GDDR5; base clock 1,165 MHz, boost 1,317 MHz; NVIDIA Corp., Santa Clara, CA, USA), and 16 GB of DDR4-2800 RAM (2×8 GB; Micron Technology Inc., Boise, ID, USA). The operating system was Ubuntu 18.04 LTS with NVIDIA driver version 481.1. Initial CNN experiments were conducted on this same platform under Keras 2.4.3 / TensorFlow 2.3.1.

Subsequent training runs were migrated to a Google Colab environment with an NVIDIA L4 GPU (24 GB GDDR6; high-memory instance) running TensorFlow 2.19.0 and Python 3.10. This transition provided sufficient GPU memory for the ResNet50-based architectures and enabled a reproducible cloud execution environment. All training runs used a fixed global random seed (`SEED = 42`) applied to NumPy, TensorFlow, and the Python hash environment.

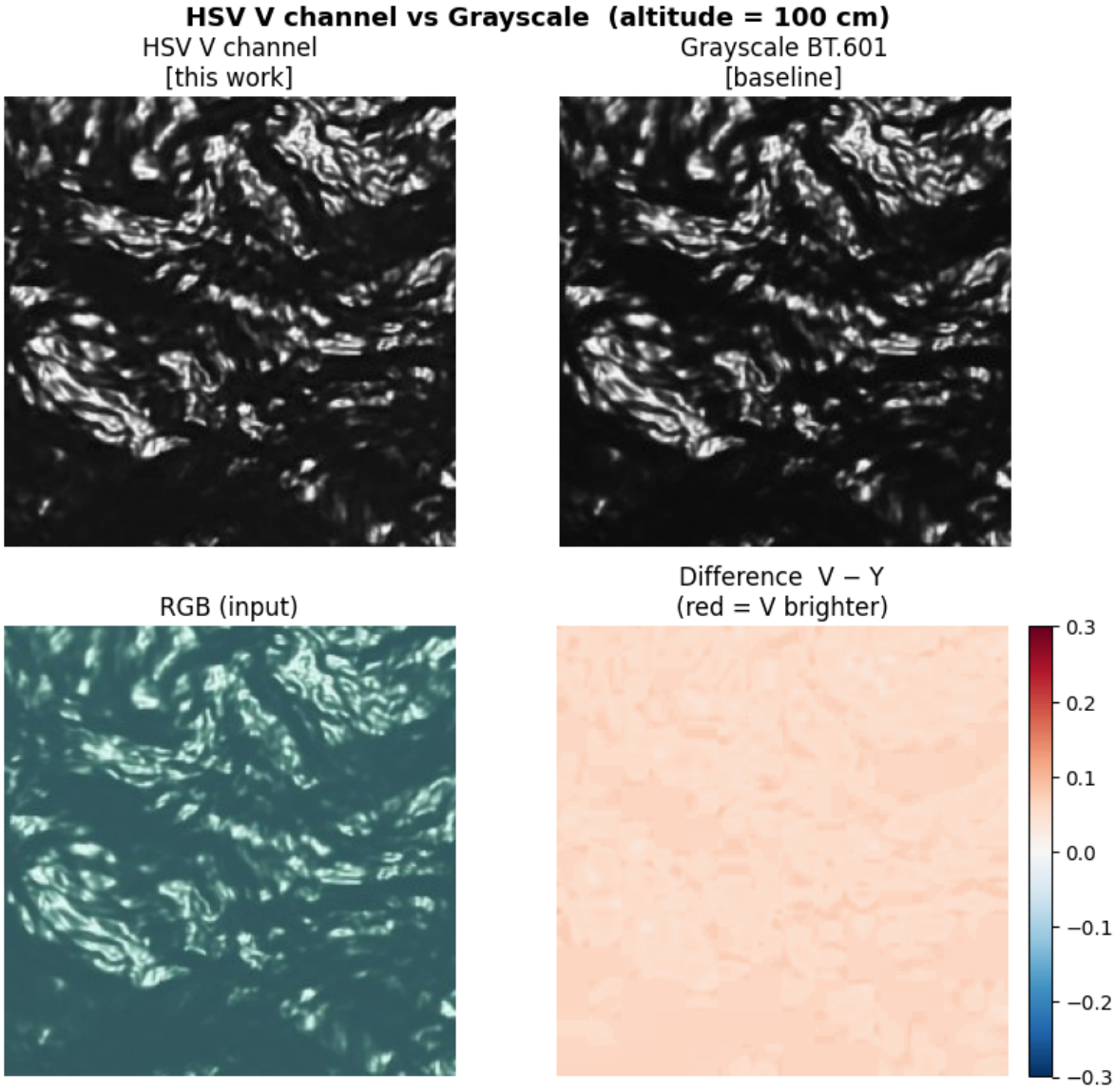

Figure 7: Side-by-side comparison of a synthetic RGB image (left) and the corresponding HSV Value channel image (right) at 100 cm altitude. Discarding hue and saturation eliminates color cast from the water surface while retaining the specular texture structure.

4.3 Feature Engineering

4.3.1 Image Input and Value Channel Extraction

Each input image is first resized to 224×224 pixels using area interpolation, which minimizes aliasing during downscaling. The RGB image is then converted to the HSV color space and the Value channel is isolated, after which pixel intensities are normalized to the range $[0, 1]$. As discussed in Section 4.1, retaining V rather than a weighted luminance (e.g., ITU-R BT.601) preserves specular highlight peaks that carry altitude-correlated texture information, while removing color variation unrelated to height.

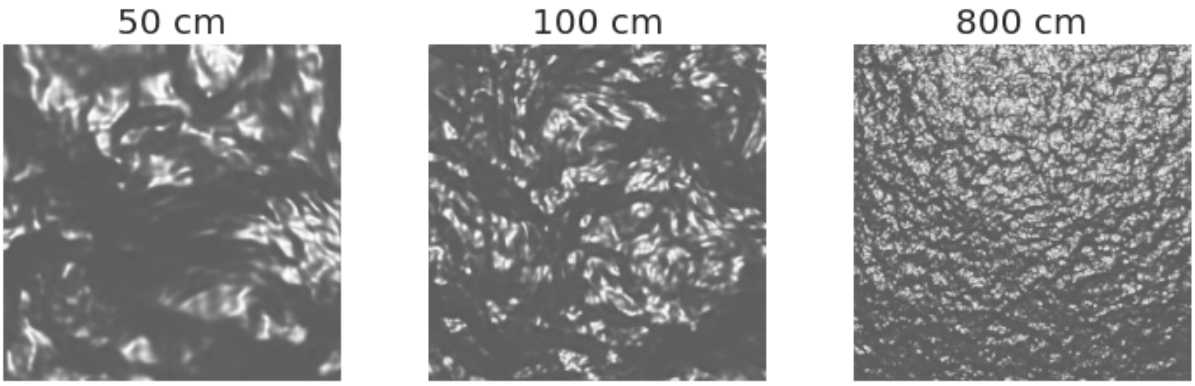

Figure 8: Synthetic Value-channel images at 50, 100, and 800 cm (left to right) with no blur applied. The progressive smoothing of surface texture with increasing altitude is clearly visible.

Figure 9: Synthetic Value-channel images at 80 cm with motion blur at different directions.

4.3.2 Preprocessed Feature Vector

For the multi-input fusion architectures, a 12-element pre-processed feature vector is extracted from the Value channel alongside the image matrix. The vector encodes pixel statistics, spectral energy distribution, gradient statistics, texture entropy, corner density, and local variance. Feature definitions and their expected monotonic relationship with altitude are summarized in Table 2. Each element was chosen in an empirical battery of tests, following a weak learning strategy, as seen in systems based on random forests or more modern tabular models like LightGBM.

4.3.2.1 FFT (`fft_energy`). The three FFT energy features (indices 4–6) are computed by partitioning the 2D log-magnitude spectrum into concentric radial frequency bands. Defining the normalized radial coordinate $r \in [0, 1]$ as the Euclidean distance from the DC component relative to the maximum possible frequency radius, the fractional energy in band k is:

Table 2: Preprocessed feature vector: index, name, description, and expected trend with *increasing* altitude. ↑ = increases, ↓ = decreases.

Idx	Name	Description	Trend
0	`mean_v`	Mean pixel intensity of the V channel	↑
1	`std_v`	Standard deviation of V channel	↓
2	`skew_v`	Skewness of intensity distribution	↓
3	`kurt_v`	Kurtosis; glints produce heavy tails at low altitude	↓
4	`fft_energy_low`	FFT energy fraction in $r < 0.20$ (large-scale / DC)	↓
5	`fft_energy_mid`	FFT energy in $0.20 \leq r < 0.60$	↑
6	`fft_energy_high`	FFT energy in $r \geq 0.60$ (fine ripples / capillary waves)	↑
7	`grad_mean`	Mean Sobel gradient magnitude	↑
8	`grad_std`	Standard deviation of Sobel gradient magnitude	↑
9	`entropy`	Normalised image entropy over a 32-bin histogram	↑
10	`shi_tomasi_count`	Shi-Tomasi corner count (glint density proxy)	↑
11	`local_std_mean`	Mean standard deviation of non-overlapping 8×8 blocks	↑

$$E_k = \frac{\sum_{(u,v) \in B_k} |\mathcal{F}(u,v)|}{\sum_{u,v} |\mathcal{F}(u,v)| + \varepsilon}, \tag{23}$$

where B_k is the set of frequency components in band k and ε is a small constant for numerical stability. At low altitudes the surface texture appears coarser, concentrating spectral energy near DC (low band); at high altitudes fine ripple patterns redistribute energy toward higher radial frequencies (high band). Fig. 10 confirms this trend empirically on the synthetic dataset, and Fig. 11 shows the corresponding increase in Shi-Tomasi corner count with altitude.

4.3.2.2 Mean Sobel gradient magnitude (`grad_mean`). The Sobel operator is applied separately along the horizontal and vertical axes of the Value channel using 3×3 kernels, and the resulting gradient magnitude map $\|\nabla V\| = \sqrt{G_x^2 + G_y^2}$ is computed at every pixel. The mean of this map measures the average edge strength across the entire image. At low altitudes, the camera resolves large individual wave facets that produce a few, well-separated intensity transitions; as altitude increases, a greater number of smaller surface features become simultaneously visible within the field of view, raising the overall

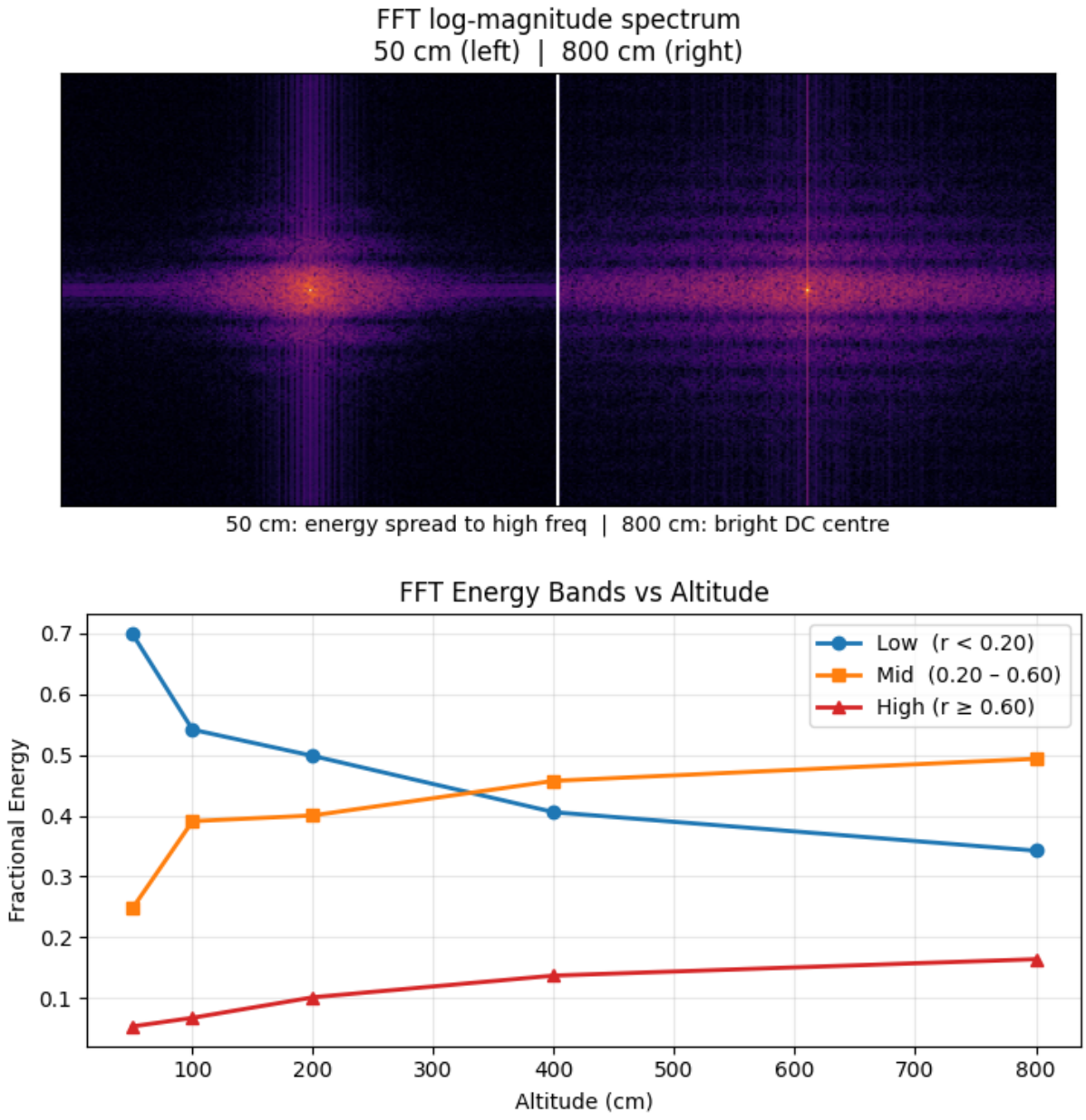

Figure 10: Left: fractional FFT energy in each radial band as a function of altitude (average over eight samples per level). As altitude increases, energy migrates from the low band to the mid and high bands, consistent with the appearance of finer ripple structures at greater distances. Right: log-magnitude spectra at 50 cm (left half) and 800 cm (right half); the 800 cm spectrum shows a broader energy distribution and a brighter DC peak.

density of edges and consequently the mean gradient magnitude.

4.3.2.3 Standard deviation of Sobel gradient magnitude (`grad_std`). Computed over the same pixel-wise gradient magnitude map as `grad_mean`, the standard deviation captures the spatial variability of edge strength rather than its average level. A low value indicates that gradient magnitudes are uniformly distributed across the image,

while a high value reflects the presence of localized high-contrast regions such as specular glint hot spots or sharp wave crests surrounded by smoother areas. This feature therefore encodes information about the heterogeneity of the surface texture, which changes with altitude as the spatial distribution of resolvable wave structures evolves.

4.3.2.4 Normalized image entropy (`entropy`). A 32-bin histogram of Value channel pixel intensities is constructed over the range $[0, 1]$, and Shannon entropy is computed as $H = -\sum_i p_i \log_2 p_i$, then divided by $\log_2 32$ to normalize it to $[0, 1]$. A small constant $\varepsilon = 10^{-10}$ is added to each bin before the logarithm evaluation to avoid undefined values. High entropy indicates a broad, uniform intensity distribution characteristic of complex, multi-scale surface textures visible at higher altitudes; low entropy indicates a peaked distribution where most pixels share similar brightness values, which occurs close to the surface where large smooth facets dominate the image.

4.3.2.5 Shi-Tomasi corner count (`shi_tomasi_count`). The Shi-Tomasi detector [52] identifies image points at which both eigenvalues of the local structure tensor exceed a quality threshold τ, indicating strong intensity gradients in two independent directions. Applied to the Value channel with a quality level of 0.5 and a minimum inter-corner distance of 24 pixels, the detector returned approximately 100 candidate points in 1000 samples. On water surfaces, specular reflection peaks create localized high-gradient regions that satisfy the corner criterion; their count therefore serves as a proxy for glint density. As altitude increases, a greater number of wave facets produce resolvable reflections within the field of view, and the detected corner count increases until it saturates at the detector ceiling.

4.3.2.6 Mean of 8×8 block standard deviations (`local_std_mean`). The Value channel image is partitioned into non-overlapping 8×8 pixel blocks, and the standard deviation of pixel intensities is computed within each block. The mean of these per-block standard deviations is then taken as a scalar feature. Unlike the global standard deviation of the full image, this measure captures local texture activity at a fixed spatial scale: a block with uniform intensity (smooth water facet) contributes near-zero variance, while a block containing a wave edge or specular glint contributes high variance. The aggregate mean therefore reflects the spatial density of active texture regions, which increases with altitude as more fine-scale surface structures fit within a single 8×8 patch.

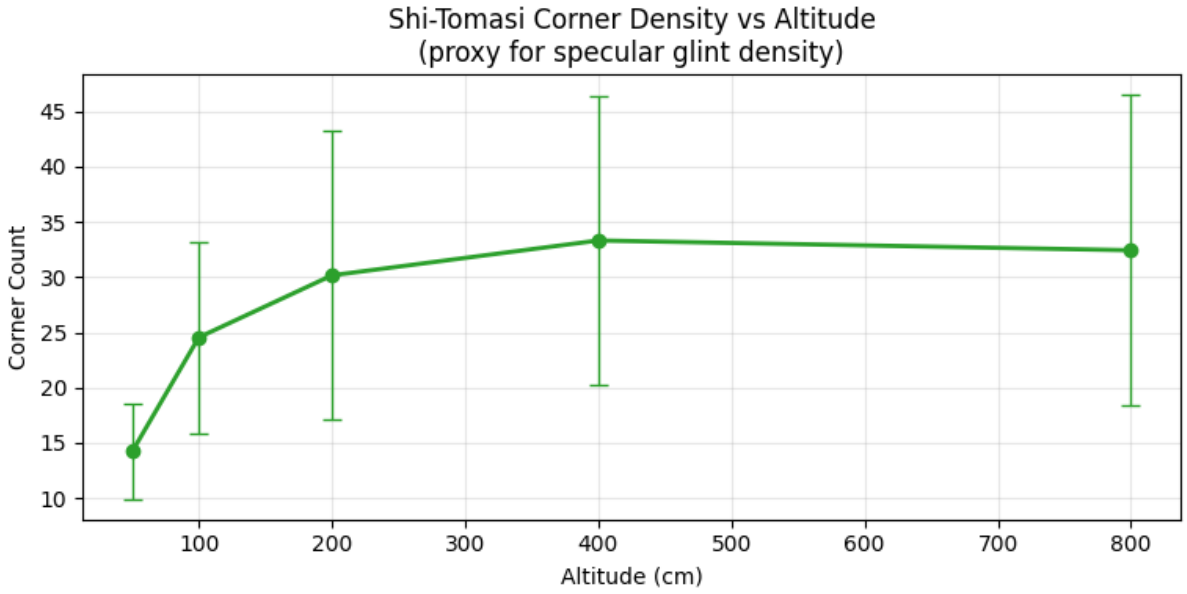

Figure 11: Shi-Tomasi corner count as a function of altitude.

4.3.3 Target Normalization and Dataset Split

All ground-truth altitude labels (in centimeters) were standardized prior to training using a `StandardScaler` fitted exclusively on the training partition

$$\tilde{y} = \frac{y - \mu_{\text{train}}}{\sigma_{\text{train}}}, \tag{24}$$

where $\mu_{\text{train}} = 288.1$ cm and $\sigma_{\text{train}} = 225.3$ cm are the mean and standard deviation of the training labels. The scaler parameters were saved alongside the model weights and applied in reverse at evaluation to recover metric-scale predictions. Standardization was preferred over linear min-max normalization because it does not bound the output range to $[0, 1]$, which is desirable for a regression head with linear activation.

The synthetic dataset (10,517 samples after balancing across altitude levels) was divided into training, validation, and test partitions using a stratified 70/15/15 split, yielding 7,365 training, 1,574 validation, and 1,578 test samples. Stratification was performed by binning altitude labels into 21 equal-width intervals (one per rendered level), ensuring proportional representation of all altitude classes in each partition.

4.4 Model Architectures and Training Protocol

Four neural network architectures were designed and evaluated within a unified training framework. All models receive the HSV Value channel as their primary image input; two of the four additionally receive a handcrafted 12-element feature vector extracted

from the same image. The following subsections describe the shared data pipeline, the individual architectures, and the training procedure.

4.4.1 WaterNet: Custom CNN Baseline

The first architecture, *WaterNet* (`WaterNet_CustomCNN`), is a VGG-inspired [34] convolutional regression network designed for the single-channel Value image. The network consists of four convolutional blocks, each with two consecutive Conv2D layers (5×5 kernels, ReLU activation, same padding) followed by MaxPooling2D (stride 2). Dropout (rate 0.3) is applied after blocks 3 and 4 to regularize the deeper feature maps. A GlobalAveragePooling2D layer replaces the flatten layer of the original VGG design, producing a 256-dimensional descriptor with far fewer parameters. A three-layer Dense regression head ($512 \rightarrow 256 \rightarrow 128$ units, ReLU, Dropout 0.3) followed by a linear output neuron completes the architecture. An optional *ClampedLinear* layer clips the normalized output to the valid range, enforcing the physical altitude constraint. The full layer sequence is given in Table 3, and the model file is 41MB in size.

Table 3: *WaterNet* architecture summary.

Layer (type)	**Output shape**	**Params**
image_input (InputLayer)	$(224, 224, 1)$	0
conv_0_a (Conv2D, 5×5, 32)	$(224, 224, 32)$	832
conv_0_b (Conv2D, 5×5, 32)	$(224, 224, 32)$	25,632
pool_0 (MaxPooling2D)	$(112, 112, 32)$	0
conv_1_a (Conv2D, 5×5, 64)	$(112, 112, 64)$	51,264
conv_1_b (Conv2D, 5×5, 64)	$(112, 112, 64)$	102,464
pool_1 (MaxPooling2D)	$(56, 56, 64)$	0
conv_2_a (Conv2D, 5×5, 128)	$(56, 56, 128)$	204,928
conv_2_b (Conv2D, 5×5, 128)	$(56, 56, 128)$	409,728
pool_2 (MaxPooling2D) + Dropout(0.3)	$(28, 28, 128)$	0
conv_3_a (Conv2D, 5×5, 256)	$(28, 28, 256)$	819,456
conv_3_b (Conv2D, 5×5, 256)	$(28, 28, 256)$	1,638,656
pool_3 (MaxPooling2D) + Dropout(0.3)	$(14, 14, 256)$	0
gap (GlobalAveragePooling2D)	(256)	0
dense_0 (Dense, 512) + Dropout(0.3)	(512)	131,584
dense_1 (Dense, 256) + Dropout(0.3)	(256)	131,328
dense_2 (Dense, 128) + Dropout(0.3)	(128)	32,896
output (Dense, 1, linear)	(1)	129
Total trainable parameters		**3,548,897**

4.4.2 WaterNet-Fusion: Multi-Input Late-Fusion Model

The second architecture, *WaterNet-Fusion* (`WaterNet_Fusion`), extends the baseline with a late-fusion branch that ingests the 12-element handcrafted feature vector of Section 4.3.2 in parallel with the image. The **image branch** is identical to WaterNet up to and including the three Dense layers ($512 \rightarrow 256 \rightarrow 128$ units), each followed by Dropout. The **feature branch** processes the 12-element vector through two Dense layers ($64 \rightarrow 32$ units, ReLU). The 128-dimensional image embedding and the 32-dimensional feature embedding are concatenated (forming a 160-dimensional vector) and passed through a fusion Dense layer (128 units, ReLU) with Dropout, before the linear regression output. A ClampedLinear layer applies the physical constraint to the final prediction.

The *WaterNet-FusionLite* variant (`WaterNet_Fusion_lite`, labeled WN-FusLT in benchmark results) uses a lighter image branch with only three convolutional blocks (32, 64, 128 filters; 3×3 kernels) and a single 64-unit embedding Dense layer, fusing a 64-dimensional image embedding with the 32-dimensional feature embedding. This reduces the total parameter count by more than ten-fold relative to WaterNet-Fusion, providing a reference for the accuracy-versus-complexity trade-off. The architecture is presented at Table 4, with 3.63 MB in size.

4.4.3 ResNet50 Transfer Learning Baseline

The third architecture, *WaterNet-ResNet50* (`WaterNet_ResNet50`), replaces the custom backbone with a ResNet50 [25] pretrained on ImageNet [15]. Because the network expects three-channel input and the Value channel is single-channel, the input tensor is expanded by concatenating the channel with itself twice (channel replication):

$$\mathbf{x}_{3\text{ch}} = \left[\, V \,\|\, V \,\|\, V \,\right] \in R^{224\times224\times3}. \tag{25}$$

The replicated tensor is then processed by `resnet50.preprocess_input`, which applies ImageNet zero-centering statistics. The backbone proceeds the same way as in a three channel input.

A two-stage fine-tuning protocol was employed. In **Stage 1**, the entire ResNet50 backbone was frozen (23,587,712 non-trainable parameters) and only the regression head was optimized; 557,569 parameters were trainable. In **Stage 2**, the top residual macroblock (`conv5_x`) was selectively unfrozen and retrained at a reduced learning rate of 10^{-5}, while BatchNorm layers were kept in inference mode to preserve the running statistics accumulated during ImageNet pretraining. The full model contains 24,145,281 parameters in total (92.11 MB).

Table 4: *WaterNet-FusionLite* architecture summary.

Layer (type)	Output shape	Params
Image branch		
image_input (InputLayer)	$(224, 224, 1)$	0
img_conv0_a (Conv2D, 3×3, 32)	$(224, 224, 32)$	320
img_conv0_b (Conv2D, 3×3, 32)	$(224, 224, 32)$	9,248
img_pool0 (MaxPooling2D)	$(112, 112, 32)$	0
img_conv1_a (Conv2D, 3×3, 64)	$(112, 112, 64)$	18,496
img_conv1_b (Conv2D, 3×3, 64)	$(112, 112, 64)$	36,928
img_pool1 (MaxPooling2D)	$(56, 56, 64)$	0
img_conv2_a (Conv2D, 3×3, 128)	$(56, 56, 128)$	73,856
img_conv2_b (Conv2D, 3×3, 128)	$(56, 56, 128)$	147,584
img_pool2 (MaxPooling2D)	$(28, 28, 128)$	0
img_gap (GlobalAveragePooling2D)	(128)	0
img_embedding (Dense, 64)	(64)	8,256
Feature branch		
feature_input (InputLayer)	(12)	0
feat_dense0 (Dense, 64)	(64)	832
feat_dense1 (Dense, 32)	(32)	2,080
Fusion head		
fusion (Concatenate)	(96)	0
fusion_dense (Dense, 128)	(128)	12,416
fusion_dropout (Dropout)	(128)	0
output (Dense, 1, linear)	(1)	129
Total trainable parameters		**310,145**

Table 5: *WaterNet-ResNet50* architecture summary.

Layer (type)	Output shape	Params
Input and channel replication		
image_input (InputLayer)	$(224, 224, 1)$	0
channel_replicate (Concatenate)	$(224, 224, 3)$	0
Backbone		
resnet50 (Functional)	$(7, 7, 2048)$	23,587,712
Regression head		
gap (GlobalAveragePooling2D)	(2048)	0
dense_0 (Dense, 256) + Dropout	(256)	524,544
dense_1 (Dense, 128) + Dropout	(128)	32,896
output (Dense, 1, linear)	(1)	129
Trainable parameters (Stage 1)		**557,569**
Non-trainable parameters (backbone)		23,587,712

4.4.4 ResNet50-Fusion: Multi-Input ResNet50 Model

The fourth architecture, *WaterNet-ResNet50-Fusion* (`WaterNet_ResNet50_MultiInput`, labeled RN50-Fus), applies the late-fusion strategy of Section 4.4.2 to the ResNet50 backbone. The **image branch** uses channel replication, `resnet50.preprocess_input`, the frozen ResNet50 backbone, and a 128-unit Dense embedding. The **feature branch** processes the 12-element vector through two Dense layers ($64 \to 32$ units, ReLU). The 128-dimensional image embedding and 32-dimensional feature embedding are concatenated (160-dimensional fusion vector), passed through a 128-unit fusion Dense layer with Dropout, and fed to the linear output. The model totals 23,873,633 parameters (91.07 MB), with 285,921 trainable in Stage 1.

Table 6: *WaterNet-ResNet50-Fusion* architecture summary.

Layer (type)	**Output shape**	**Params**
Image branch		
image_input (InputLayer)	$(224, 224, 1)$	0
channel_replicate (Concatenate)	$(224, 224, 3)$	0
resnet50 (Functional)	$(7, 7, 2048)$	23,587,712
img_gap (GlobalAveragePooling2D)	(2048)	0
img_embedding (Dense, 128)	(128)	262,272
Feature branch		
feature_input (InputLayer)	(12)	0
feat_dense0 (Dense, 64)	(64)	832
feat_dense1 (Dense, 32)	(32)	2,080
Fusion head		
fusion (Concatenate)	(160)	0
fusion_dense (Dense, 128)	(128)	20,608
fusion_dropout (Dropout)	(128)	0
output (Dense, 1, linear)	(1)	129
Trainable parameters (Stage 1)		**285,921**
Non-trainable parameters (backbone)		23,587,712

A consolidated overview of all four architectures is provided in Table 7.

4.4.5 Loss Function and Optimizer

All models were compiled with the Huber loss:

$$L_\delta(y, \hat{y}) = \begin{cases} \dfrac{1}{2}(y - \hat{y})^2 & \text{if } |y - \hat{y}| \le \delta, \\ \delta\left(|y - \hat{y}| - \dfrac{\delta}{2}\right) & \text{otherwise,} \end{cases} \tag{26}$$

Table 7: Summary of the four model architectures evaluated. Trainable parameters refer to Stage 1 (head-only training) for the ResNet50-based models; all parameters are trainable for the custom-backbone models.

Model	Backbone	Feature	Total params	Trainable
WaterNet	**Custom CNN (VGG-style)**	**No**	**3,548,897**	**3,548,897**
WN-FusLT	WaterNet	Yes	310,145	310,145
ResNet50	ResNet50 (ImageNet)	No	24,145,281	557,569
RN50-Fus	ResNet50 (ImageNet)	Yes	23,873,633	285,921

with $\delta = 1.0$ (in normalized target units). The Huber loss provides quadratic gradients near zero error (as in MSE) while remaining linearly bounded for large residuals (as in MAE), which is desirable given the presence of challenging frames (specular glare and two types of blurring) in the dataset.

The AdamW optimizer was used for all models, with initial learning rate $\eta = 10^{-3}$, weight decay $\lambda = 0.01$, and gradient clipping by global norm (`clipnorm` = 1.0). For the head-only Stage 1 of the ResNet50-based models the initial learning rate was set to $\eta = 10^{-3}$, consistent with the custom-backbone models, and reduced to $\eta = 10^{-5}$ for Stage 2 backbone fine-tuning.

4.4.6 Training Callbacks

A standard callback stack was applied to all training runs:

- **EarlyStopping**: monitors validation loss; restores best weights after 12 epochs without improvement.
- **ReduceLROnPlateau**: halves the learning rate after 5 consecutive epochs without improvement in validation loss, down to a minimum of 10^{-7}.
- **ModelCheckpoint**: saves the checkpoint with the lowest validation loss.
- **TensorBoard**: records per-epoch scalar metrics and weight histograms for post-hoc analysis.

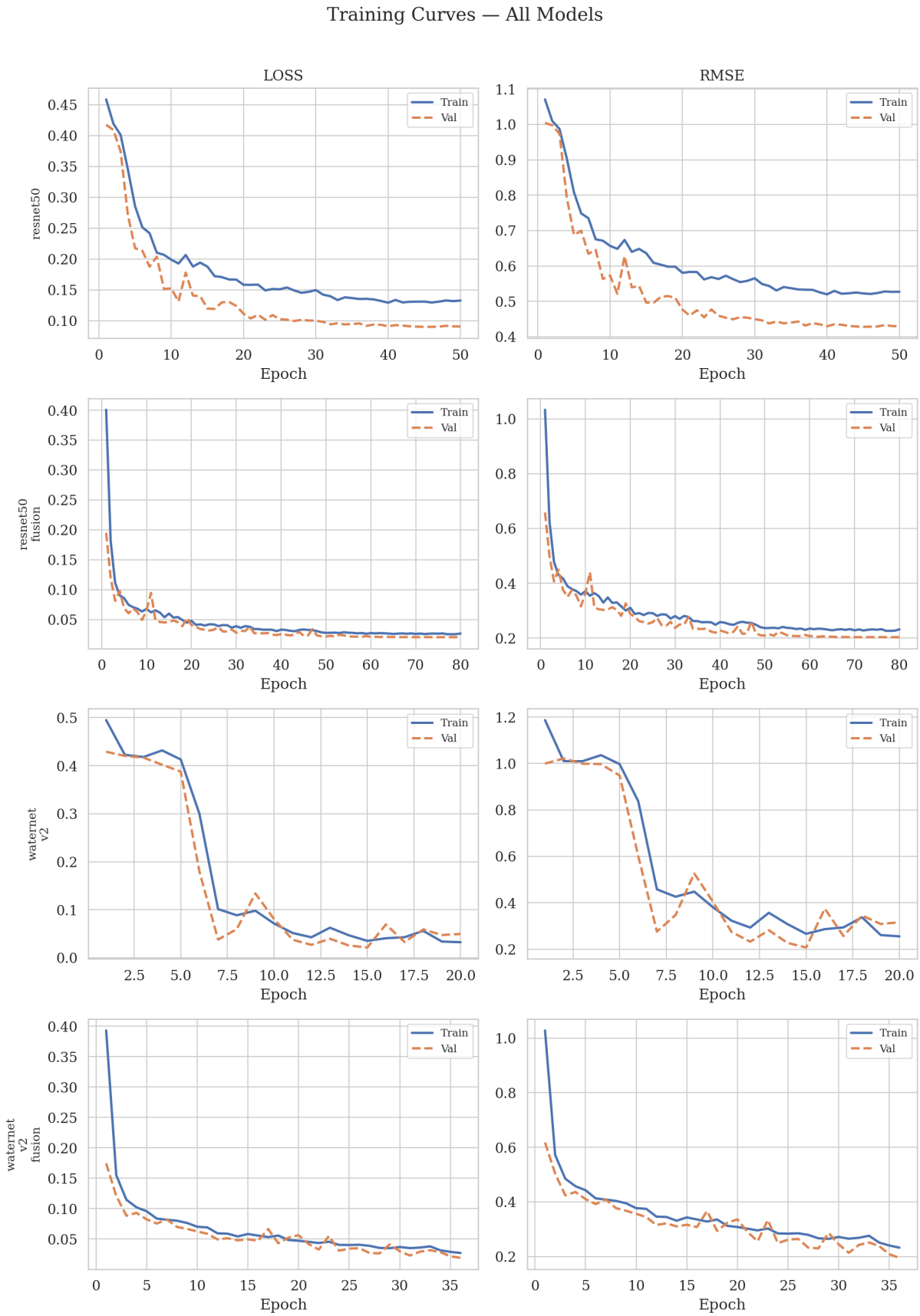

Figure 12: Training curves for all models, with loss and error function.

5 RESULTS AND DISCUSSION

This chapter presents the experimental results obtained from the four neural network architectures described in Section 4.4, evaluated first on the held-out synthetic test partition under Subsection 5.1, and then on a real-world dataset acquired from a human piloted UAV in a coastal site in Subsection 5.2. Subsection 5.3 provides a comparative analysis of the architectures in terms of size, complexity, and practical deployment considerations. The synthetic and real datasets, trained CNN models, Blender models and used algorithms are available at the repository `https://github.com/jvitorsn/waternet`.

5.1 Synthetic Test Set Evaluation

The synthetic test partition (2,630 samples, stratified across 21 altitude levels from 50 to 800 cm) was used to evaluate each model's ability to regress altitude from Value-channel images drawn from the same domain as the training data. This evaluation isolates the model's learning capacity from domain-shift effects, providing an upper bound on expected accuracy before real-world deployment.

Fig. 12 shows the training and validation curves in Huber loss and RMSE for all four architectures. The ResNet50-based models (top two rows) required more epochs to converge than the WaterNet variants (bottom two rows), consistent with the larger number of parameters in the regression head being optimized against a frozen backbone. The ResNet50 baseline (top row) exhibits a persistent gap between training and validation loss, indicating mild overfitting to the synthetic training distribution. In contrast, the ResNet50-Fusion model (second row) achieves tighter convergence between the two curves, suggesting that the handcrafted feature branch acts as a regularizer by providing complementary altitude-correlated signals that reduce the model's reliance on potentially spurious visual patterns.

The WaterNet baseline (third row) converges rapidly within approximately 10 epochs and reaches the lowest final validation loss among all models, despite having significantly fewer parameters than the ResNet50 variants. The WaterNet-FusionLite model (bottom row) shows similarly fast convergence with slightly higher validation loss, reflecting the trade-off introduced by the lighter convolutional backbone (three blocks instead of four, with 3×3 kernels instead of 5×5).

5.1.1 Per-Model Prediction Analysis

To examine prediction behavior in detail, Figs. 15–13 display the predicted altitude against the ground truth for every sample in the synthetic test set, sorted by ascending

true altitude. These plots reveal how each model tracks the staircase pattern formed by the 21 discrete altitude levels.

5.1.1.1 WaterNet (Fig. 13). The custom CNN achieves the tightest tracking of the ground-truth staircase among all models. Predictions cluster closely around the true altitude at all levels, with notably low variance in the 50–300 cm range. Beyond 400 cm, the spread increases moderately but remains smaller than that of the ResNet50 variants. The strong performance of this comparatively shallow architecture suggests that the altitude estimation task over water surfaces is well served by texture-level features captured in the early convolutional layers, and that the deeper, more abstract representations learned by ResNet50 may not provide additional benefit for this specific domain.

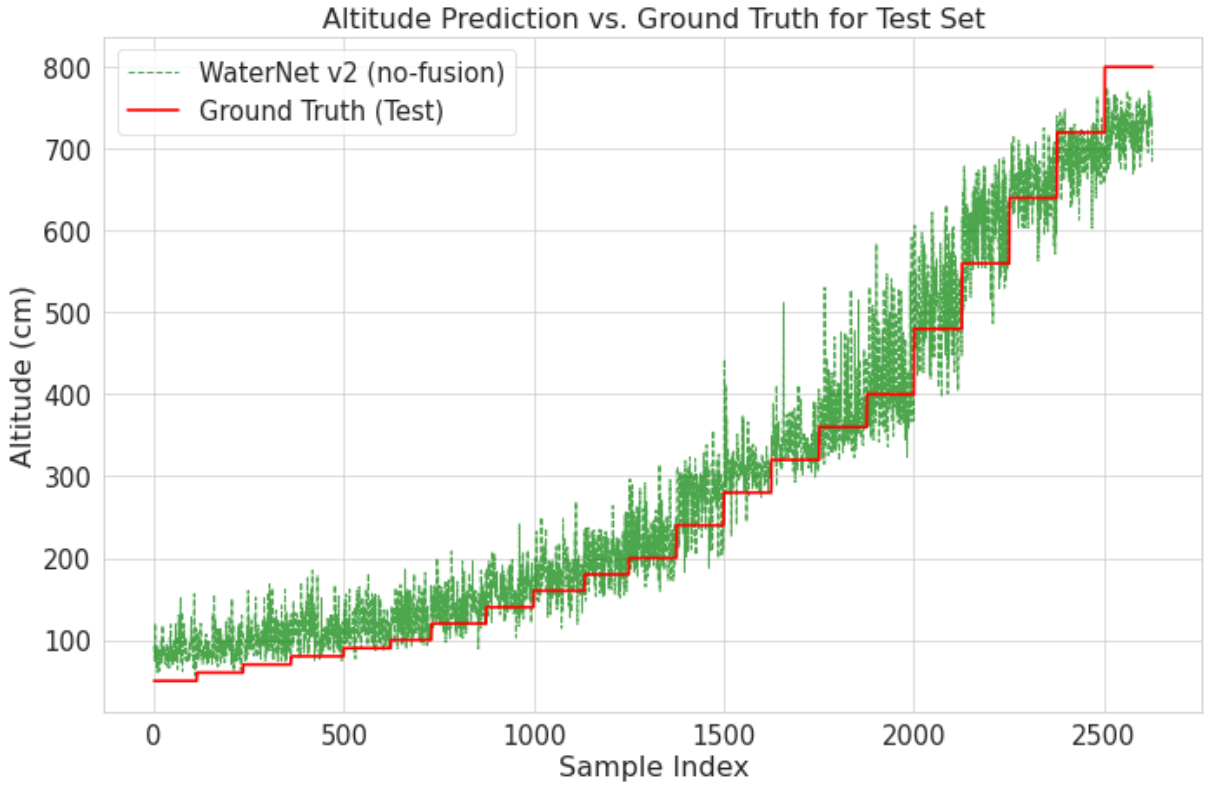

Figure 13: Altitude prediction (green) vs. ground truth (red) for the synthetic test set using the WaterNet baseline.

5.1.1.2 WaterNet-FusionLite (Fig. 14). The lightweight fusion model achieves prediction quality comparable to the full WaterNet, with tight tracking at low-to-mid altitudes and slightly increased spread at the highest levels. The fact that this model uses only three convolutional blocks and 3×3 kernels, with roughly one-tenth the parameters of the full WaterNet, makes it a strong candidate for embedded deployment where computational resources are constrained.

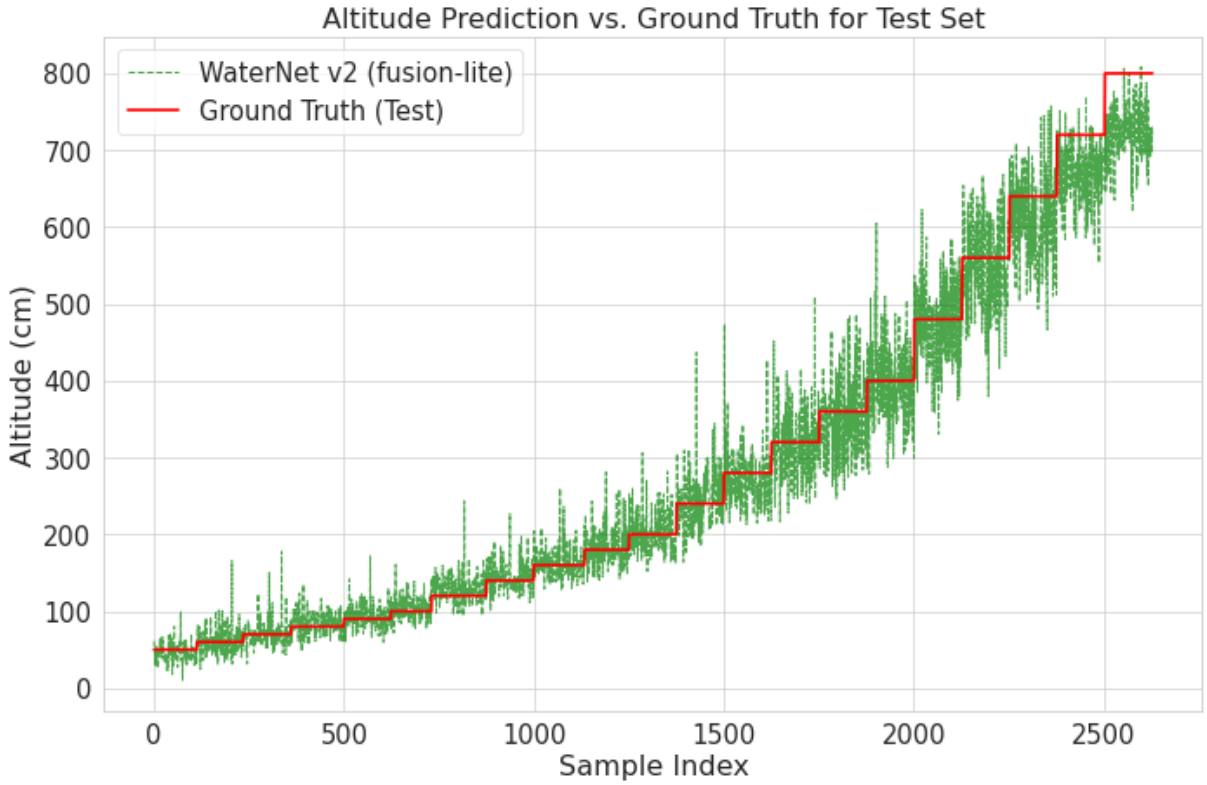

Figure 14: Altitude prediction vs. ground truth for the synthetic test set using the WaterNet-FusionLite model.

5.1.1.3 ResNet50 (Fig. 15). The ResNet50 baseline produces high variance at nearly all altitude levels, with predictions scattering widely around the ground truth. At low altitudes (50–100 cm), predictions oscillate between approximately 50 and 200 cm, and the spread increases substantially beyond 400 cm, where individual predictions deviate by up to 300 cm from the true value. This behavior is consistent with the relatively high validation loss observed during training and suggests that the frozen ImageNet features, while powerful for natural image recognition, do not efficiently encode the fine-grained texture differences that distinguish adjacent altitude levels in water surface imagery.

5.1.1.4 ResNet50-Fusion (Fig. 16). The addition of the 12-element handcrafted feature vector produces a marked improvement in prediction stability. The predictions track the ground-truth staircase more closely, particularly in the 50–400 cm range, where the variance is substantially reduced compared to the ResNet50 baseline. At higher altitudes (above 500 cm), some spread remains, but the tracking of the altitude trend is preserved. This improvement indicates that the FFT energy ratios, gradient statistics, and Shi-Tomasi corner counts provide altitude-discriminative information that complements the CNN's learned representations.

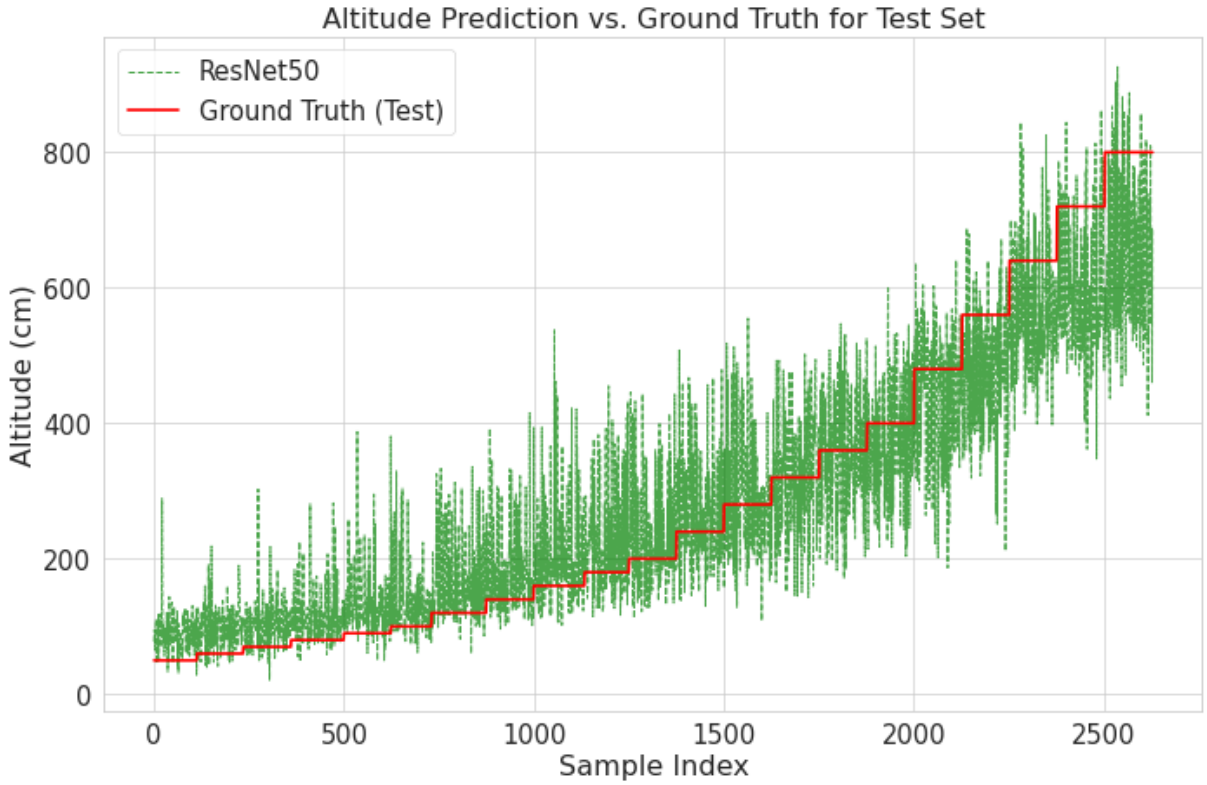

Figure 15: Altitude prediction (green) vs. ground truth (red) for the synthetic test set using the ResNet50 baseline.

5.1.2 Error Distribution Analysis

Figs. 19–18 present histograms of the prediction error (predicted minus true altitude, in centimeters) for each model on the synthetic test set, overlaid with a fitted Gaussian curve to characterize the error distribution.

5.1.2.1 WaterNet (Fig. 17). The WaterNet error distribution shows a slight positive bias ($\mu = 18.7$ cm) with a standard deviation of $\sigma = 40.3$ cm. The distribution is right-skewed, with a heavier tail toward positive errors (overestimation). This systematic overestimation bias is consistent with the observation also noted in the other models and may originate from the brightness distribution mismatch between synthetic and real samples, where synthetic images at low altitudes tend to be slightly brighter than their real-world counterparts.

5.1.2.2 WaterNet-FusionLite (Fig. 18). The lightweight fusion model achieves the most centered error distribution among the WaterNet variants ($\mu = -7.6$ cm, $\sigma = 41.7$ cm). The slight negative bias suggests a tendency toward mild underestimation. The distribution shape is approximately symmetric with moderate tails, comparable to those of the ResNet50-Fusion model.

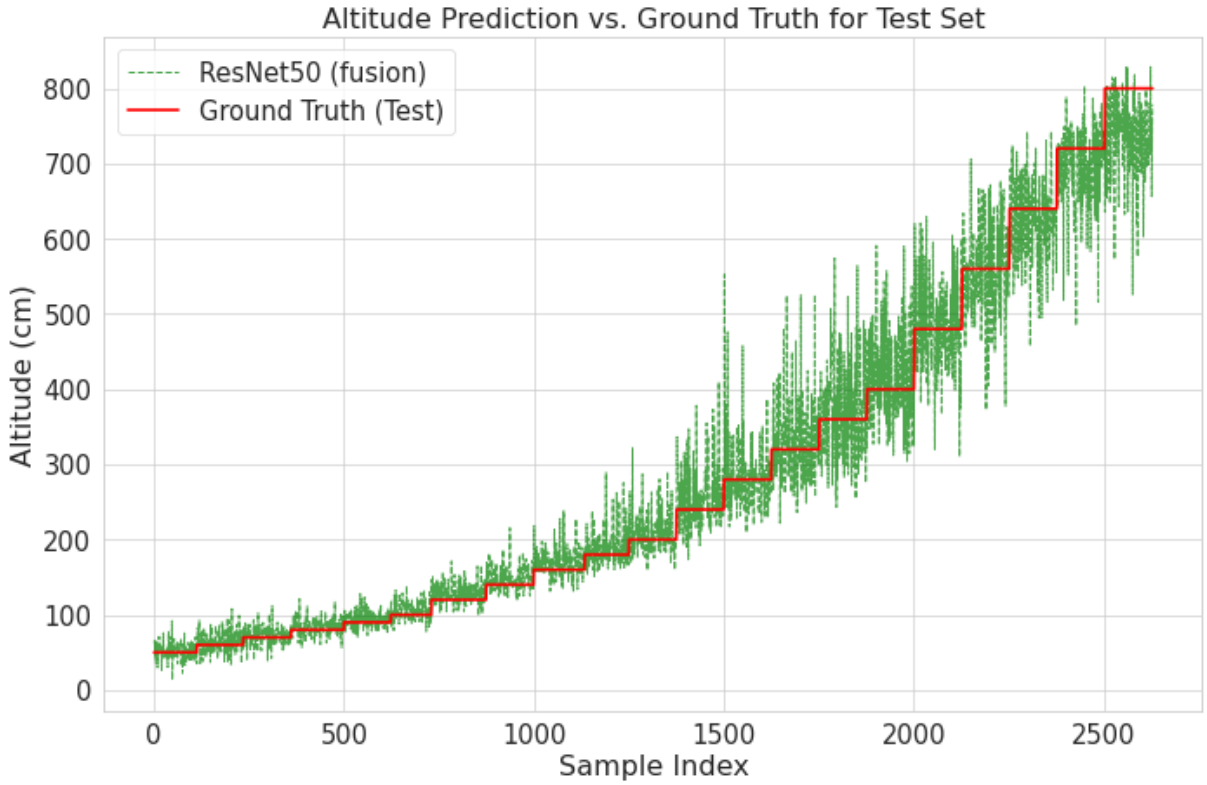

Figure 16: Altitude prediction (green) vs. ground truth (red) for the synthetic test set using the ResNet50-Fusion model.

5.1.2.3 ResNet50 (Fig. 19). The error distribution is approximately centered near zero ($\mu = -1.7$ cm) but exhibits a large standard deviation ($\sigma = 98.1$ cm), with tails extending beyond ± 300 cm. The Gaussian fit captures the central tendency reasonably well, although the histogram shows slightly heavier tails than a true Gaussian would predict, indicating the presence of outlier predictions.

5.1.2.4 ResNet50-Fusion (Fig. 21). The fusion model substantially narrows the error distribution ($\mu = 0.8$ cm, $\sigma = 44.3$ cm), reducing the standard deviation by more than half compared to the baseline. The distribution is approximately symmetric and more sharply peaked, indicating that the majority of the inferences fall within ± 50 cm of the true altitude. The tail extent is also reduced, with errors rarely exceeding ± 200 cm.

5.2 Real-World Dataset Evaluation

The real-world evaluation dataset consists of 4,100 frames extracted from video captured at a coastal site, spanning an altitude range of approximately 0.8 to 12.6 m as measured by a fusion of barometric altimeter and GPS readings. A LiDAR sensor was also present during data collection and is included as a reference sensor for comparison. Importantly, none of the real images were used during training; this evaluation therefore

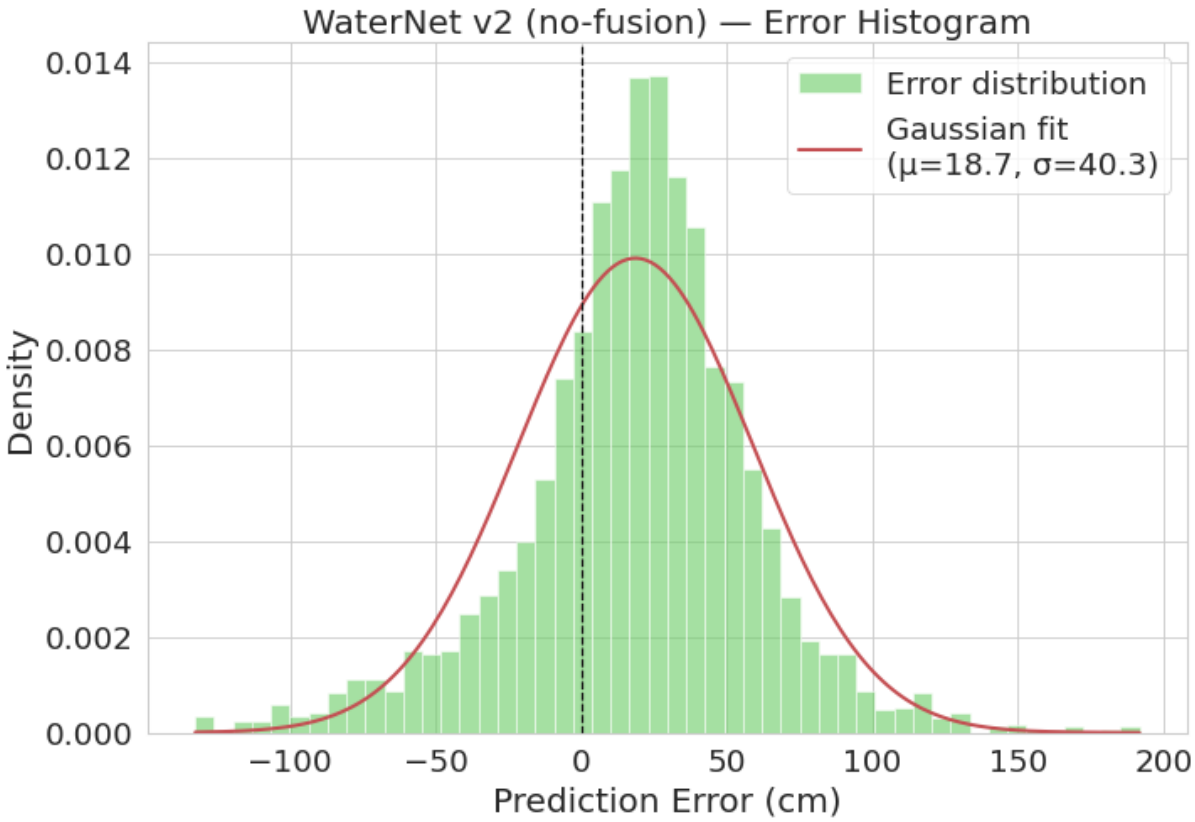

Figure 17: Error histogram for the WaterNet baseline on the synthetic test set. The fitted Gaussian has $\mu = 18.7$ cm and $\sigma = 40.3$ cm.

measures the sim-to-real generalization capability of each model.

It must be noted that ground-truth altitude measurements are themselves subject to uncertainty. The barometric-GPS fusion sensor is influenced by atmospheric pressure variations, humidity gradients, and GPS multipath effects near water surfaces. The LiDAR sensor, as discussed in Section 5.2.3, proved unreliable over water due to specular reflection causing frequent zero-returns and extreme outliers (Fig. **??**). Consequently, the error metrics reported below should be interpreted as upper bounds on the true model error, since the ground truth itself carries measurement noise.

The inference plot for WaterNet and WaterNet-FusLT is presented in Fig. 22. For ResNet50 and ResNet50-Fus, the inferences are shown in Fig. 23.

5.2.1 Aggregate Performance Metrics

Table 8 summarizes the performance of all four CNN models and the LiDAR sensor on the real-world dataset. Among the CNN models, WaterNet achieves the lowest MAE (1.787 m) and RMSE (2.337 m), with an R^2 of 0.551, which is the highest among all evaluated methods. The WaterNet-FusionLite model ranks second in MAE (2.208 m) and RMSE (2.859 m), followed by the ResNet50-based architectures. The ResNet50-Fusion

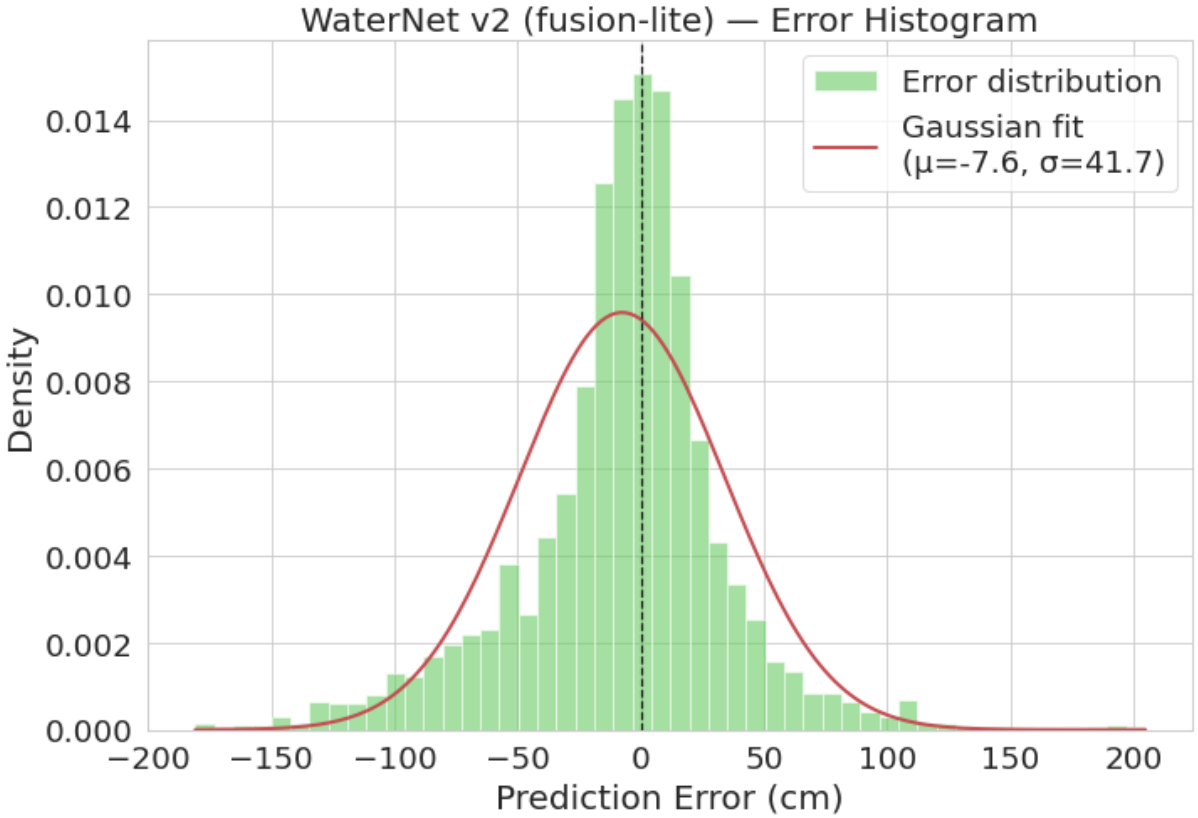

Figure 18: Error histogram for the WaterNet-FusionLT model on the synthetic test set. The fitted Gaussian has $\mu = -7.6$ cm and $\sigma = 41.7$ cm.

model achieves the lowest absolute bias (0.118 m) among all CNN models, indicating that its predictions are nearly centered around the ground truth on average, despite having a higher error magnitude than WaterNet.

Table 8: Performance metrics on the real-world dataset (4,100 frames). All error values are in meters unless otherwise indicated. Best CNN result per metric is shown in bold.

Model	MAE	RMSE	Bias	R^2	MAPE	MedAE	P95	Max	Err.σ
ResNet50	2.700	3.591	−1.455	−0.061	62.82%	2.054	7.985	16.516	3.285
RN50-Fus	2.824	3.266	**0.118**	0.122	105.11%	2.939	6.002	12.986	3.264
WaterNet	**1.787**	**2.337**	0.518	**0.551**	**57.50%**	**1.373**	**4.875**	**8.680**	**2.277**
WN-FusLT	2.208	2.859	1.192	0.327	79.54%	1.669	5.760	12.705	2.590
LiDAR	1.697	4.364	−1.509	−0.567	24.78%	0.050	9.700	77.105	4.096

The LiDAR achieves the lowest MAE overall (1.697 m) but the highest RMSE (4.364 m) among all methods, a diagnostic pattern indicating the presence of extreme outliers. This large gap between MAE and RMSE is characteristic of heavy-tailed error distributions: the LiDAR produces accurate readings for a subset of frames (reflected in its low median absolute error of 0.050 m) but returns severely erroneous measurements

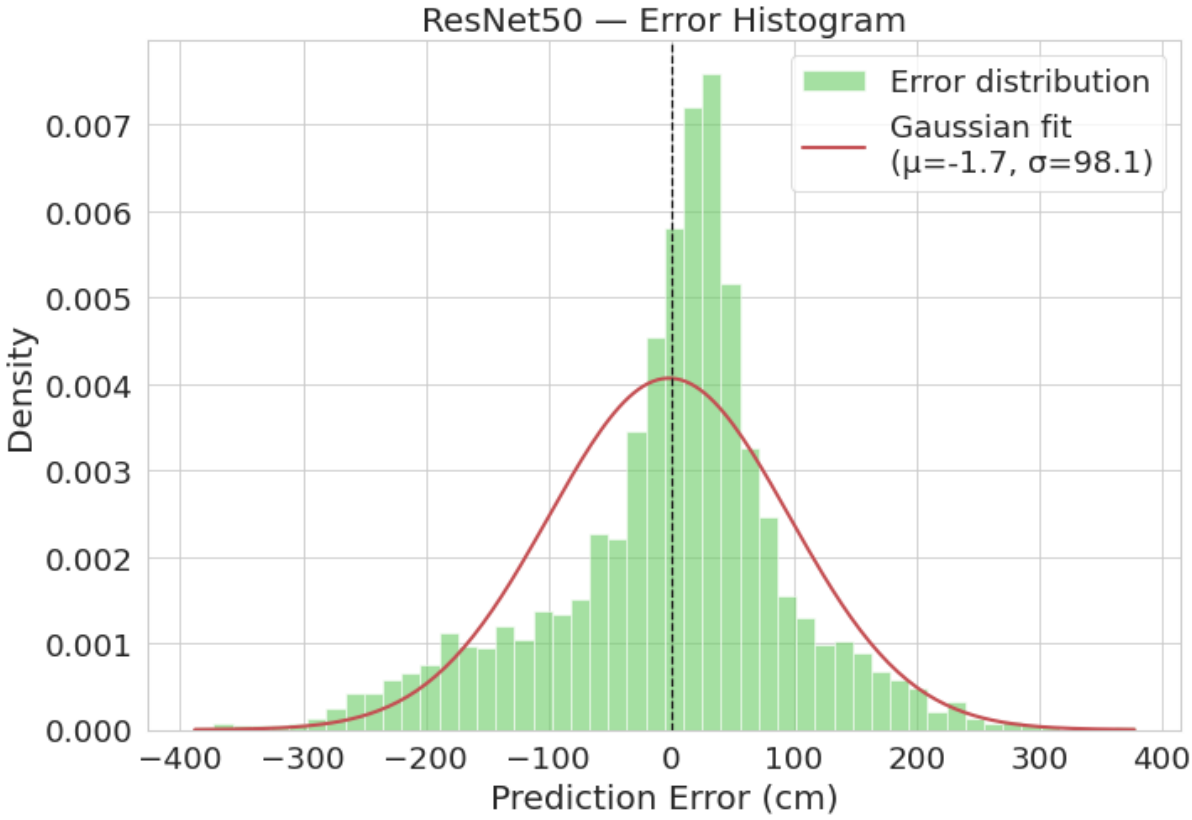

Figure 19: Error histogram for the ResNet50 baseline on the synthetic test set. The fitted Gaussian has $\mu = -1.7$ cm and $\sigma = 98.1$ cm.

in others, inflating the squared-error penalty. In practical terms, a sensor with the lowest MAE but the highest RMSE is unreliable for safety-critical altitude control, where worst-case performance matters more than average accuracy.

5.2.2 Systematic Bias Analysis

Fig. 24 shows the systematic bias (mean signed error) for each model. The ResNet50 baseline exhibits a strong negative bias (−1.455 m), meaning it systematically underestimates the true altitude. This underestimation is consistent with the saturation behavior observed where ResNet50 predictions plateau around 4–5 m regardless of the actual altitude. The LiDAR sensor shows a similar negative bias (−1.509 m), driven by its frequent near-zero returns over the water surface.

In contrast, the WaterNet models exhibit positive biases (+0.518 m for WaterNet, +1.192 m for WN-FusLT), indicating systematic overestimation. This upward shift is consistent with the observation from early experiments that the synthetic training domain produces images with slightly different brightness statistics than the real environment, particularly at low altitudes where synthetic images appear more gray (high floor value) than their real-world counterparts.

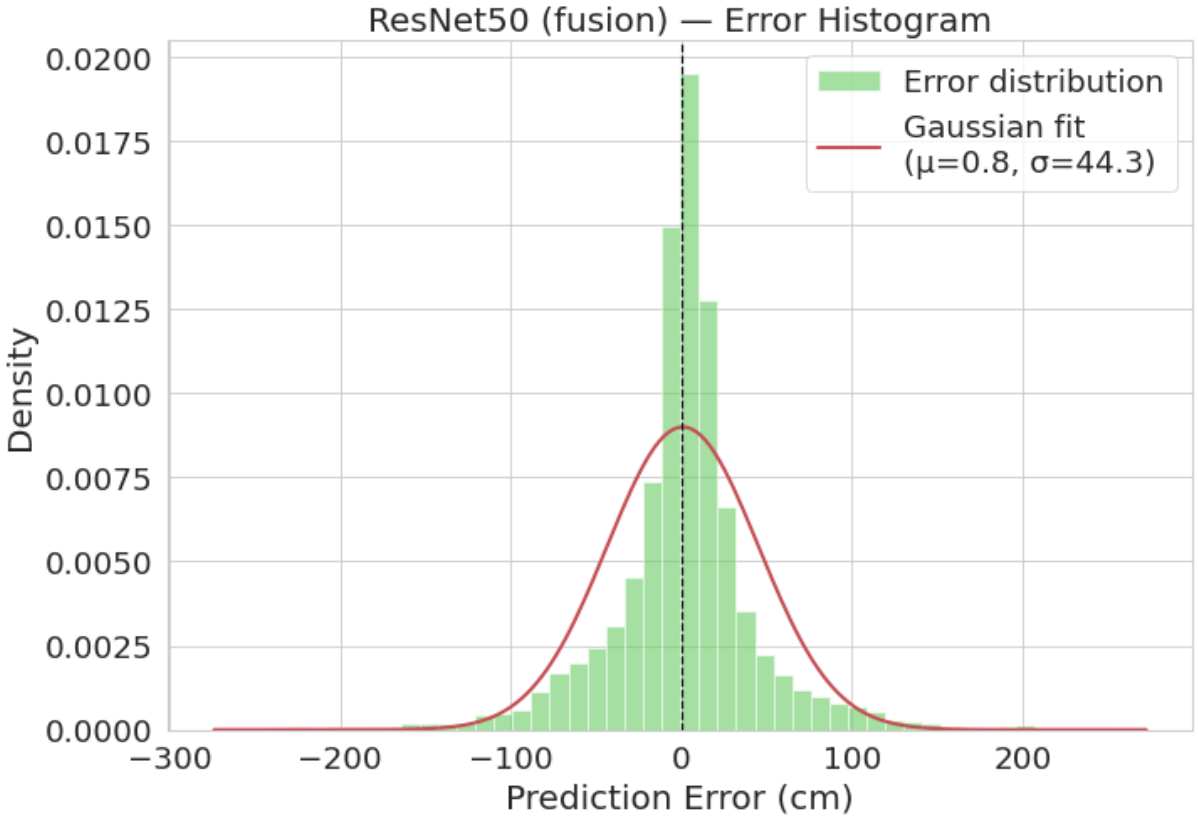

Figure 20: Error histogram for the ResNet50-Fusion model on the synthetic test set. The fitted Gaussian has $\mu = 0.8$ cm and $\sigma = 44.3$ cm.

The ResNet50-Fusion model achieves the most centered predictions, with a bias of only +0.118 m. This near-zero bias suggests that the handcrafted features, which are computed directly from the image statistics rather than learned from the synthetic domain, help anchor the predictions and partially compensate for the domain shift affecting the visual features.

5.2.3 LiDAR Sensor Analysis

The LiDAR sensor was included in the evaluation as a reference for the conventional approach to altitude measurement. However, the results confirm the fundamental limitation described in the introduction: LiDAR is unreliable over water surfaces. Of the 4,100 frames, 857 (20.9%) produced near-zero readings (below 0.1 m), indicating complete signal loss due to specular reflection or absorption at the water surface. At the other extreme, 6 frames produced readings exceeding 20 m, with a maximum of 77.1 m, likely caused by multi path reflections or sensor saturation. These failure modes explain the paradoxical combination of the lowest MAE (1.697 m, driven by the many frames where the sensor works correctly) and the highest RMSE (4.364 m, driven by the catastrophic outliers) among all evaluated methods. The negative R^2 value (-0.567) further confirms

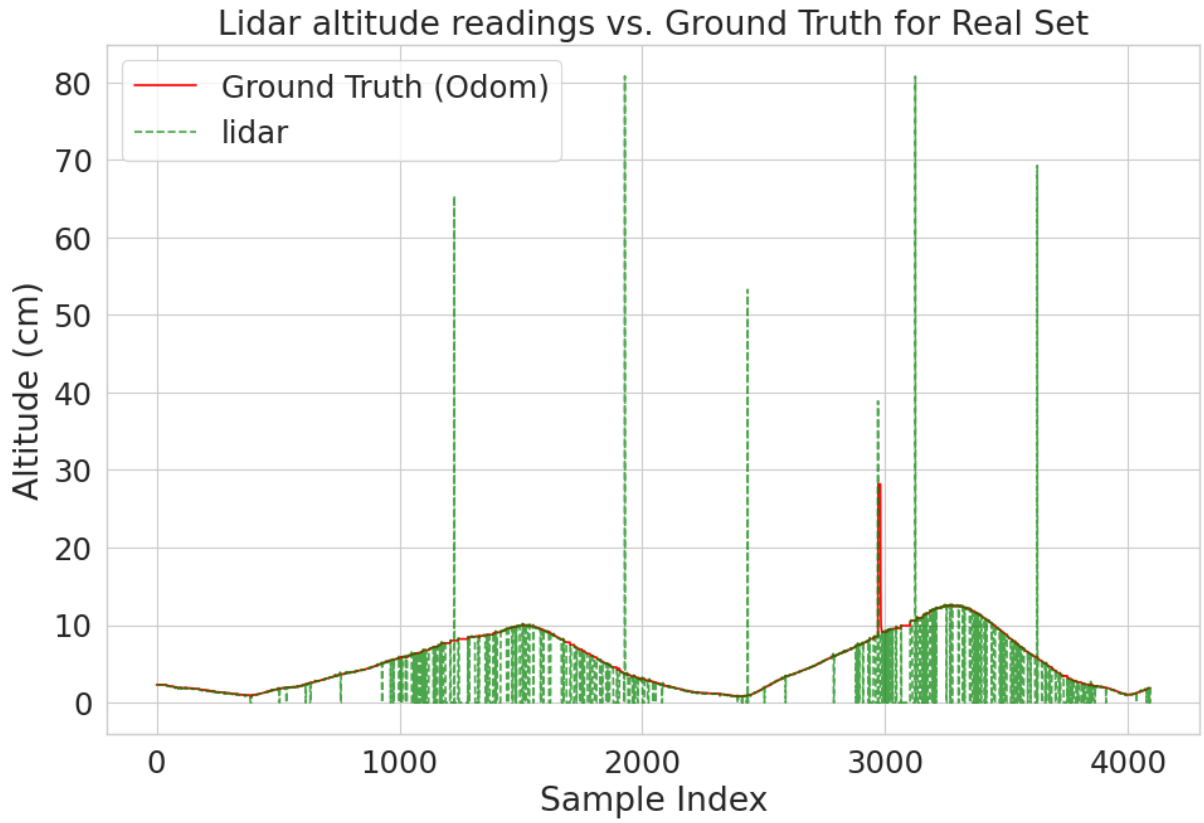

Figure 21: LiDAR readings (green) vs ground truth (red) for the real dataset

that the LiDAR performs worse than a constant mean predictor over this dataset.

This result validates the central motivation of the present work: over water surfaces, conventional ranging sensors can fail catastrophically and unpredictably, producing errors of an order of magnitude larger than the true altitude. A vision-based estimator, even with moderate average error, provides substantially more predictable and bounded behavior, which is a desirable property for flight control systems that rely on altitude estimates for safety-critical decisions.

5.3 Model Comparison

Table 9 provides a consolidated comparison of the four architectures across dimensions relevant to both research evaluation and practical deployment.

5.3.1 Effect of Transfer Learning vs. Domain-Specific Training

A counterintuitive finding of this evaluation is that the custom WaterNet architecture, trained from scratch on synthetic water surface imagery, outperforms the ResNet50-based models that leverage ImageNet-pretrained weights. This result contrasts with the

Table 9: Comprehensive comparison of the four evaluated architectures. Synthetic metrics refer to the test partition.

Property	**ResNet50**	**RN50-Fus**	**WaterNet**	**WN-FusLT**
Architecture				
Backbone	ResNet50	ResNet50	Custom (VGG)	Custom (VGG)
Kernel size	$3 \times 3/1 \times 1$	$3 \times 3/1 \times 1$	5×5	3×3
Conv. blocks	16 (residual)	16 (residual)	4	3
Feature fusion input	No	Yes (12-dim)	No	Yes (12-dim)
Size and complexity				
Total parameters	24.1 M	23.9 M	3.5 M	310 K
Trainable (Stage 1)	558 K	286 K	3.5 M	310 K
Pretrained backbone	ImageNet	ImageNet	None	None
Training behavior				
Convergence	$\sim$50 ep.	$\sim$80 ep.	$\sim$20 ep.	$\sim$35 ep.
Overfitting tendency	Moderate	Low	Low	Low
Synthetic test performance				
Error μ (cm)	-1.7	$+0.8$	$+18.7$	-7.6
Error σ (cm)	98.1	44.3	40.3	41.7
Real-world performance				
MAE (m)	2.700	2.824	**1.787**	2.208
RMSE (m)	3.591	3.266	**2.337**	2.859
Bias (m)	-1.455	**0.118**	0.518	1.192
R^2	-0.061	0.122	**0.551**	0.327
Deployment considerations				
Edge suitability	Low	Low	Moderate	High

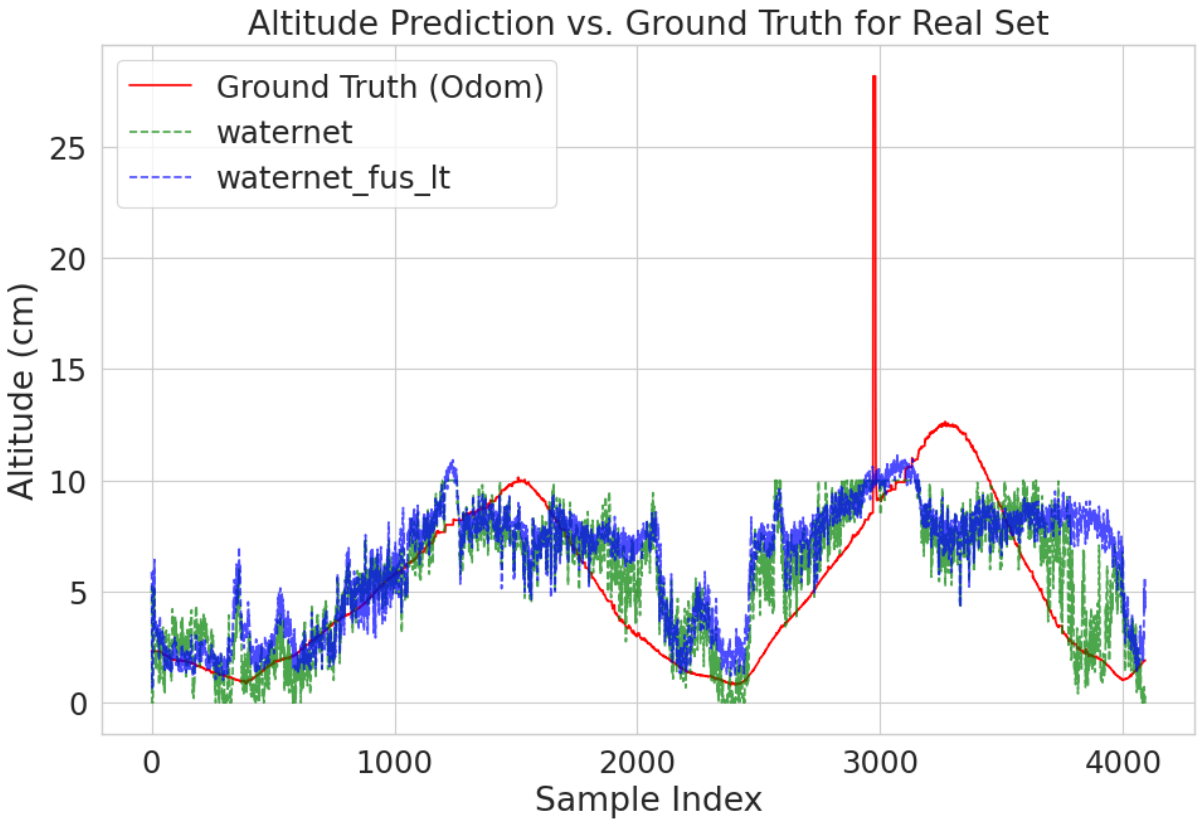

Figure 22: WaterNet inference (green) vs WaterNet-FusLT inference (blue) vs ground truth (red) for the real dataset

common assumption in computer vision that transfer learning from large-scale datasets universally improves performance, and deserves careful interpretation.

The ResNet50 backbone was pretrained on ImageNet, a dataset dominated by natural images of objects, animals, and scenes with rich semantic content and well-defined structural features. The features learned from such data, like edges, textures, shapes, and object parts, are optimized for classification tasks where the goal is to distinguish between semantically different categories. In contrast, altitude estimation over water requires discriminating between images of the same visual category (water surface) based on subtle differences in texture scale, brightness distribution, and specular reflection density. These differences are encoded in low-level and mid-level features that may be overridden or compressed by the deep representations learned for classification.

The WaterNet architecture, with its four convolutional blocks of 5×5 kernels, is specifically designed to capture large receptive fields at moderate depth, which aligns well with the spatial scale of the altitude-dependent texture features. The 5×5 kernels in the early layers capture local wave patterns and brightness gradients more directly than the 3×3 kernels used throughout ResNet50, where equivalent receptive fields require stacking multiple layers and traversing residual connections.

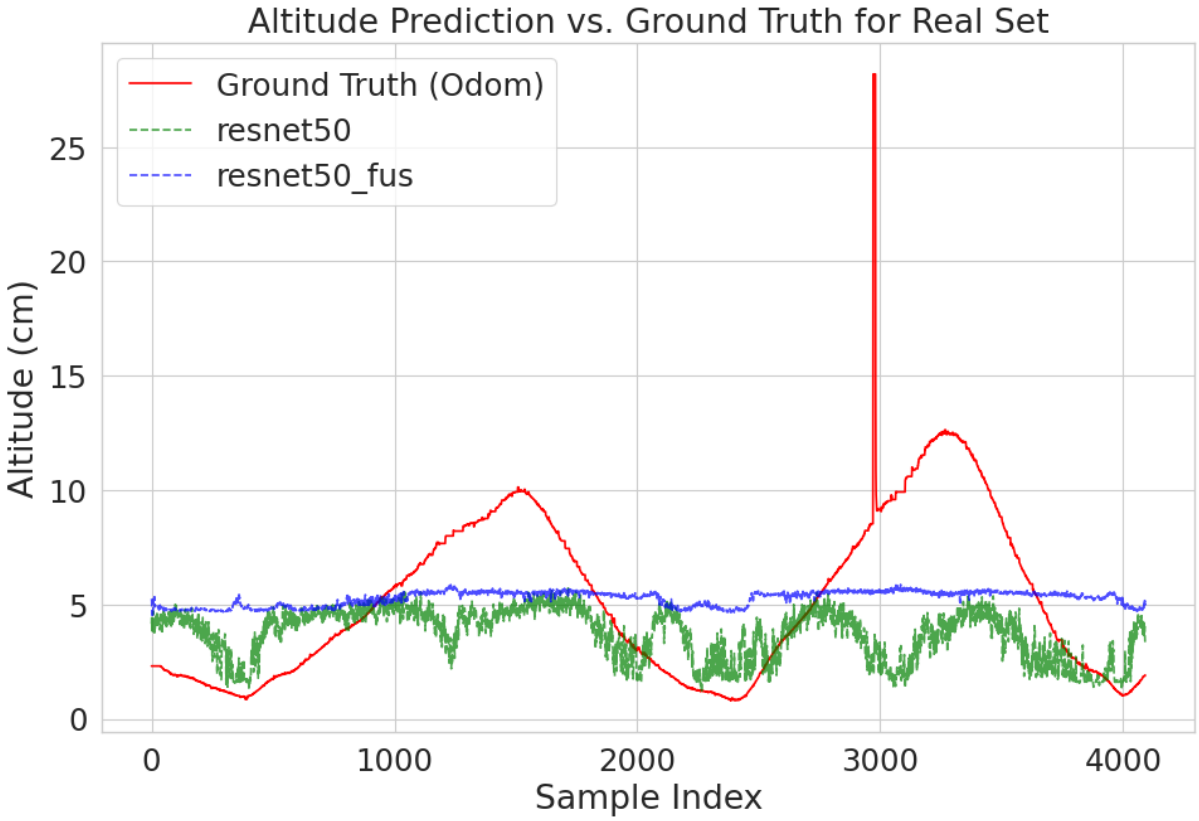

Figure 23: ResNet50 inference (green) vs ResNet50-Fus inference (blue) vs ground truth (red) for the real dataset

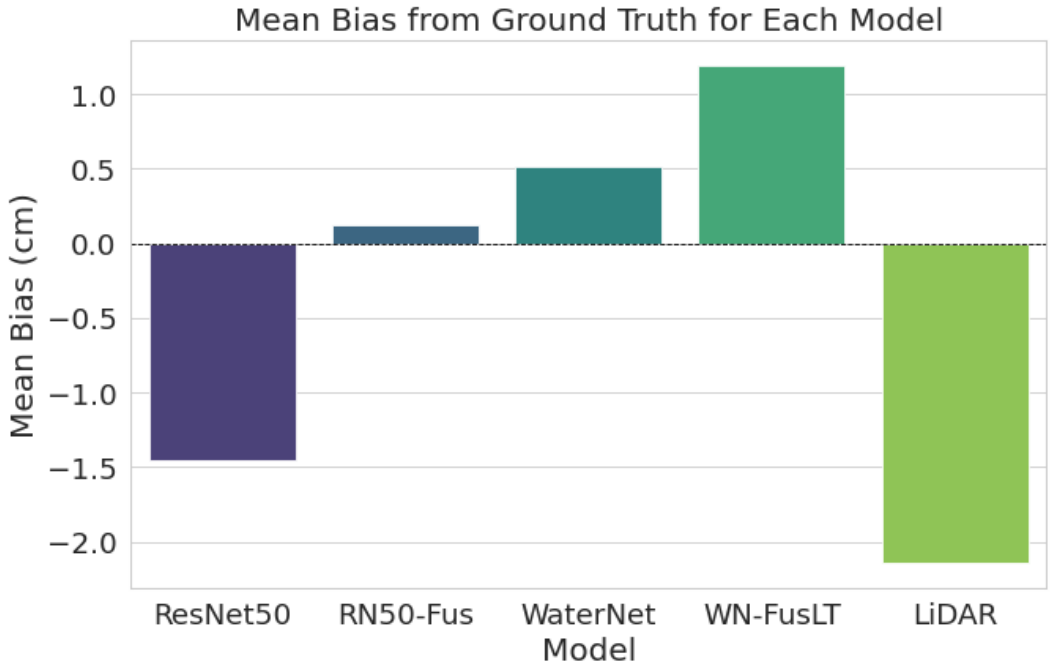

Figure 24: Systematic bias (mean signed error) per model and LiDAR sensor. Positive values indicate overestimation; negative values indicate underestimation.

5.3.2 Effect of Feature Fusion

The impact of the handcrafted feature vector is asymmetric across the two backbone families. For the ResNet50 family, fusion produces a dramatic improvement: it reduces the error standard deviation from 98.1 to 44.3 cm on the synthetic test set and shifts the real-world bias from -1.455 to $+0.118$ m. This improvement indicates that the handcrafted features provide altitude-correlated information that the frozen ResNet50 backbone fails to extract from the image alone, effectively compensating for the backbone's domain mismatch.

For the WaterNet family, fusion produces a more nuanced effect. The WaterNet-FusionLite model achieves a lower bias magnitude (-7.6 vs. $+18.7$ cm on synthetic; $+1.192$ vs. $+0.518$ m on real) but higher MAE and RMSE than the standalone WaterNet. This suggests that the handcrafted features help center the inferences but introduce additional variance, possibly because the lighter convolutional backbone (3 blocks, 3×3 kernels) produces less discriminative image embeddings that the fusion head cannot fully compensate for.

6 CONCLUSION

The central hypothesis that a CNN can learn to extract altitude-correlated visual cues from monocular images of water surfaces, in a manner analogous to human visual distance estimation was validated by the experimental results. The WaterNet baseline achieved an MAE of 1.787 m and an R^2 of 0.551 on real-world data, despite having been trained entirely on synthetic imagery. Critically, all four CNN models produced lower RMSE than the onboard LiDAR sensor (2.337–3.591 m vs. 4.364 m), which suffered from frequent signal loss (20.9% near-zero readings) and catastrophic outliers (maximum error of 77.1 m) due to the specular and semi-transparent properties of the water surface. But the ResNet50 based models have very low explicability score to be considered instead of the custom made WaterNet. This result confirms the practical motivation of the work: over water, a vision-based estimator provides predictable and bounded error behavior, and can suit as an input for flight control over water.

An unexpected finding was that the custom WaterNet architecture, trained from scratch on domain-specific data, outperformed the ResNet50-based models that leveraged ImageNet-pretrained features. This suggests that for tasks requiring discrimination based on subtle texture-scale and brightness-distribution differences within a single visual category, domain-specific feature learning with appropriately sized convolutional kernels (5×5) is more effective than transferring deep representations optimized for semantic classification across thousands of object categories. The handcrafted feature vector proved most beneficial when paired with the ResNet50 backbone, where it reduced the error standard deviation by more than half and shifted the real-world bias from -1.455 m to $+0.118$ m, effectively compensating for the backbone's domain mismatch. The WaterNet-FusionLite variant, with only 310,K parameters, demonstrated that a competitive accuracy–complexity trade-off is achievable, making embedded deployment on constrained UAV platforms a realistic prospect.

The work also revealed persistent challenges that must be addressed before operational deployment. A systematic bias was observed across all models, originating from the brightness distribution mismatch between synthetic and real domains: synthetic images at low altitudes appear slightly brighter than their real-world counterparts, and this discrepancy propagates through the Value-channel preprocessing into the learned representations. Furthermore, prediction variance increases substantially at higher altitudes, where the visual differences between adjacent altitude levels become increasingly subtle as wave texture averages out. The ground-truth measurements themselves carry non-negligible uncertainty from the barometric-GPS fusion sensor, which limits the interpretability of the reported error metrics as true model accuracy.

6.1 Future Work

Several directions are identified for extending this research toward a deployable system. First, the sim-to-real domain gap should be addressed through brightness histogram matching between synthetic and real training samples, and through domain adaptation techniques or further domain randomization of rendering parameters (lighting angle, atmospheric haze, water turbidity). Second, the systematic bias observed in all models motivates the development of a post-processing calibration stage, such as spline-based or piecewise-linear correction fitted on a small set of real-world calibration samples. Third, the real-world dataset should be expanded with measurements from diverse water bodies (rivers, lakes, reservoirs) under varying weather and illumination conditions, using a calibrated radar altimeter as ground truth rather than the barometric-GPS fusion sensor employed in this study. Fourth, model explainability should be strengthened through Grad-CAM analysis on the CNN models and deeper analysis on the feature fusion branches, to verify that the learned attention patterns correspond to physically meaningful regions of the water surface. Fifth, the deployment pipeline should be implemented as a ROS 2 node with TensorRT or ONNX quantization for real-time inference on embedded platforms such as the NVIDIA Jetson Orin Nano, including latency profiling and integration with the flight controller's altitude fusion filter. Also, a multi-output branch can help with a classification task, inferring if the subject is readable as water or not, to flag the controller to avoid using the measures outside the water surface and possibly confusing the sensor fusion filter.

Bibliography

[1] D. Acharya, K. Khoshelham, and S. Winter. Bim-posenet: Indoor camera localisation using a 3d indoor model and deep learning from synthetic images. *ISPRS Journal of Photogrammetry and Remote Sensing*, 150:245–258, 2019. ISSN 0924-2716. doi: https://doi.org/10.1016/j.isprsjprs.2019.02.020. URL `https://www.sciencedirect.com/science/article/pii/S0924271619300589`.

[2] G. B. Airy. *Tides and Waves*, volume 3. London, 1841.

[3] Blender Foundation. Ocean modifier — Blender manual, 2024. URL `https://docs.blender.org/manual/en/latest/modeling/modifiers/physics/ocean.html`.

[4] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

[5] A. A. Cabrera-Ponce and J. Martinez-Carranza. Onboard CNN-based processing for target detection and autonomous landing for MAVs. In *Lecture Notes in Computer Science*, pages 195–208. Springer International Publishing, 2020. doi: 10.1007/978-3-030-49076-8_19. URL `https://doi.org/10.1007/978-3-030-49076-8_19`.

[6] P. Chakravarty, K. Kelchtermans, T. Roussel, S. Wellens, T. Tuytelaars, and L. Van Eycken. Cnn-based single image obstacle avoidance on a quadrotor. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6369–6374, 2017. doi: 10.1109/ICRA.2017.7989752.

[7] J.-Y. Chang and Z.-X. Xu. Enhanced water puddle segmentation and detection using DCU-Net. *IEEE Internet of Things Journal*, 2024. doi: 10.1109/JIOT.2024.3466390.

[8] J. Chen and H. Liu. Laboratory water surface elevation estimation using image-based convolutional neural networks. *Ocean Engineering*, 248:110819, 2022. ISSN 0029-8018. doi: https://doi.org/10.1016/j.oceaneng.2022.110819. URL `https://www.sciencedirect.com/science/article/pii/S0029801822002633`.

[9] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.

[10] J. W. Cooley and J. W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965. doi: 10.1090/S0025-5718-1965-0178586-1.

[11] A. D. D. Craik. The origins of water wave theory. *Annual Review of Fluid Mechanics*, 36:1–28, 2004. doi: 10.1146/annurev.fluid.36.050802.122118.

[12] X.-Z. Cui, Q. Feng, S.-Z. Wang, and J.-H. Zhang. Monocular depth estimation with self-supervised learning for vineyard unmanned agricultural vehicle. *Sensors*, 22(3), 2022. ISSN 1424-8220. doi: 10.3390/s22030721. URL https://www.mdpi.com/1424-8220/22/3/721.

[13] S. Dai and Y. Wu. Motion from blur. In *2008 IEEE Conference on Computer Vision and Pattern Recognition*, pages 1–8, 2008. doi: 10.1109/CVPR.2008.4587582.

[14] E. Darles, B. Crespin, D. Ghazanfarpour, and J.-C. Gonzato. A survey of ocean simulation and rendering techniques in computer graphics. *Computer Graphics Forum*, 30(1):43–60, 2011. doi: 10.1111/j.1467-8659.2010.01828.x.

[15] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009. doi: 10.1109/CVPR.2009.5206848.

[16] M. Denninger, M. Sundermeyer, D. Winkelbauer, D. Olefir, T. Hodan, Y. Zidan, M. Elbadrawy, M. Knauer, H. Katam, and A. Lodhi. BlenderProc2: A procedural pipeline for photorealistic rendering. *Journal of Open Source Software*, 8(82):4901, 2023. doi: 10.21105/joss.04901.

[17] H. V. Dudukcu, M. Taskiran, and N. Kahraman. Uav sensor data applications with deep neural networks: A comprehensive survey. *Engineering Applications of Artificial Intelligence*, 123(Part C):106476, 2023. doi: 10.1016/j.engappai.2023.106476.

[18] D. Eigen, C. Puhrsch, and R. Fergus. Depth map prediction from a single image using a multi-scale deep network. In *Advances in Neural Information Processing Systems 27 (NeurIPS 2014)*, pages 2366–2374, 2014.

[19] M. G. Ferrick. Analysis of river wave types. *Water Resources Research*, 21(2): 209–220, 1985. doi: https://doi.org/10.1029/WR021i002p00209. URL https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1029/WR021i002p00209.

[20] J. Gao, P. Li, Z. Chen, and J. Zhang. A survey on deep learning for multimodal data fusion. *Neural Computation*, 32(5):829–864, 2020. doi: 10.1162/neco_a_01273.

[21] M. Gao, C. H. Hugenholtz, T. A. Fox, M. Kucharczyk, T. E. Barchyn, and P. R. Nesbit. Weather constraints on global drone flyability. *Scientific Reports*, 11(1), June 2021. doi: 10.1038/s41598-021-91325-w. URL https://doi.org/10.1038/s41598-021-91325-w.

[22] R. C. Gonzalez and R. E. Woods. *Digital Image Processing*. Pearson, 4th edition, 2018. ISBN 978-0-13-335672-4.

[23] R. M. Haralick, K. Shanmugam, and I. Dinstein. Textural features for image classification. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-3(6):610–621, 1973. doi: 10.1109/TSMC.1973.4309314.

[24] C. Harris and M. Stephens. A combined corner and edge detector. In *Proceedings of the 4th Alvey Vision Conference*, pages 147–151, 1988. doi: 10.5244/C.2.23.

[25] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition, 2015. URL `https://arxiv.org/abs/1512.03385`.

[26] D. Hendrycks and T. Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv: 1903.12261.

[27] T. O. Hodson. Root-mean-square error (RMSE) or mean absolute error (MAE): When to use them or not. *Geoscientific Model Development*, 15(14):5481–5487, 2022. doi: 10.5194/gmd-15-5481-2022.

[28] C. J. Horvath. Empirical directional wave spectra for computer graphics. In *Proceedings of the 2015 Symposium on Digital Production (DigiPro '15)*, pages 29–39. ACM, 2015. doi: 10.1145/2791261.2791267.

[29] X. Hua, X. Wang, T. Rui, F. Shao, and D. Wang. Light-weight uav object tracking network based on strategy gradient and attention mechanism. *Knowledge-Based Systems*, 224:107071, 2021. ISSN 0950-7051. doi: https://doi.org/10.1016/j.knosys.2021.107071. URL `https://www.sciencedirect.com/science/article/pii/S0950705121003348`.

[30] A. Joglekar, D. Joshi, R. Khemani, S. Nair, and S. Sahare. Depth estimation using monocular camera. 2011. URL `https://api.semanticscholar.org/CorpusID:15694290`.

[31] H. R. V. Joze, A. Shaban, M. L. Iuzzolino, and K. Koishida. MMTM: Multimodal transfer module for CNN fusion. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13286–13296, 2020. doi: 10.1109/CVPR42600.2020.01330.

[32] K. D. Julian, S. Sharma, J.-B. Jeannin, and M. J. Kochenderfer. Verifying aircraft collision avoidance neural networks through linear approximations of safe regions, 2019. URL `https://arxiv.org/abs/1903.00762`.

[33] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 3146–3154, 2017.

[34] V. Khandelwal. The architecture and implementation of VGG-16, 2021. URL https://pub.towardsai.net/the-architecture-and-implementation-of-vgg-16-b050e5a5920b.

[35] A. Kouris and C.-S. Bouganis. Learning to fly by myself: A self-supervised cnn-based approach for autonomous navigation. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1–9, 2018. doi: 10.1109/IROS.2018.8594204.

[36] Z. Lu and Y. Chen. Self-supervised monocular depth estimation on water scenes via specular reflection prior, 2024. URL https://arxiv.org/abs/2404.07176.

[37] G. A. Mastin, P. A. Watterberg, and J. F. Mareda. Fourier synthesis of ocean scenes. *IEEE Computer Graphics and Applications*, 7(3):16–23, 1987. doi: 10.1109/MCG.1987.276961.

[38] K. McGuire, G. de Croon, C. De Wagter, K. Tuyls, and H. Kappen. Efficient optical flow and stereo vision for velocity estimation and obstacle avoidance on an autonomous pocket drone. *IEEE Robotics and Automation Letters*, 2(2):1070–1076, 2017. doi: 10.1109/LRA.2017.2658940.

[39] M. A. Nielsen. Neural networks and deep learning, 2018. URL http://neuralnetworksanddeeplearning.com/.

[40] L. Noci, K. Roth, G. Bachmann, S. Nowozin, and T. Hofmann. Disentangling the roles of curation, data-augmentation and the prior in the cold posterior effect. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 12738–12748. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper/2021/file/6a12d7ebc27cae44623468302c47ad74-Paper.pdf.

[41] T. Oktay, H. Celik, and I. Turkmen. Maximizing autonomous performance of fixed-wing unmanned aerial vehicle to reduce motion blur in taken images. *Proceedings of the Institution of Mechanical Engineers, Part I: Journal of Systems and Control Engineering*, 232(7):857–868, Mar. 2018. doi: 10.1177/0959651818765027. URL https://doi.org/10.1177/0959651818765027.

[42] Y. Pei, Y. Huang, Q. Zou, X. Zhang, and S. Wang. Effects of image degradation and degradation removal to CNN-based image classification. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(4):1239–1253, 2021. doi: 10.1109/TPAMI.2019.2950923.

[43] J.-F. Pekel, A. Cottam, N. Gorelick, and A. S. Belward. High-resolution mapping of global surface water and its long-term changes. *Nature*, 540:418–422, 2016. doi: 10.1038/nature20584.

[44] Y. Peng, Z. Tang, G. Zhao, G. Cao, and C. Wu. Motion blur removal for uav-based wind turbine blade images using synthetic datasets. *Remote Sensing*, 14(1), 2022. ISSN 2072-4292. doi: 10.3390/rs14010087. URL https://www.mdpi.com/2072-4292/14/1/87.

[45] O. M. Phillips. The equilibrium range in the spectrum of wind-generated waves. *Journal of Fluid Mechanics*, 4(4):426–434, 1958. doi: 10.1017/S0022112058000550.

[46] M. Potmesil and I. Chakravarty. Modeling motion blur in computer-generated images. In *Proceedings of the 10th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '83)*, pages 389–399, 1983. doi: 10.1145/800059.801169.

[47] M. G. Prasad, S. Chandran, and M. S. Brown. A motion blur resilient fiducial for quadcopter imaging. In *2015 IEEE Winter Conference on Applications of Computer Vision*, pages 254–261, 2015. doi: 10.1109/WACV.2015.41.

[48] U. Rajapaksha, R. Jayasundara, R. Nawaratne, S. Ahamed, and S. Mahindaratne. Deep learning-based depth estimation methods from monocular image and videos: A comprehensive survey. *ACM Computing Surveys*, 2024. doi: 10.1145/3674505.

[49] P. S. Rajpura, R. S. Hegde, and H. Bojinov. Object detection using deep cnns trained on synthetic images. *CoRR*, abs/1706.06782, 2017. URL http://arxiv.org/abs/1706.06782.

[50] F. Roland, N. F. Caraco, J. J. Cole, and P. del Giorgio. Rapid and precise determination of dissolved oxygen by spectrophotometry: Evaluation of interference from color and turbidity. *Limnology and Oceanography*, 44(4):1148–1154, 1999. doi: https://doi.org/10.4319/lo.1999.44.4.1148. URL https://aslopubs.onlinelibrary.wiley.com/doi/abs/10.4319/lo.1999.44.4.1148.

[51] H. Samaranayake, O. Mudannayake, D. Perera, P. Kumarasinghe, C. Suduwella, K. De Zoysa, and P. Wimalaratne. Detecting water in visual image streams from uav with flight constraints. *Journal of Visual Communication and Image Representation*, 96:103933, 2023. ISSN 1047-3203. doi: https://doi.org/10.1016/j.jvcir.2023.103933. URL https://www.sciencedirect.com/science/article/pii/S1047320323001839.

[52] J. Shi and C. Tomasi. Good features to track. In *1994 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 593–600, 1994. doi: 10.1109/CVPR.1994.323794.

[53] R. Shwartz-Ziv and A. Armon. Tabular data: Deep learning is not all you need. *Information Fusion*, 81:84–90, 2022. doi: 10.1016/j.inffus.2021.11.011.

[54] T. Sieberth, R. Wackrow, and J. H. Chandler. Motion blur disturbs — the influence of motion-blurred images in photogrammetry. *The Photogrammetric Record*, 29(148): 434–453, 2014. doi: 10.1111/phor.12082.

[55] T. Sieberth, R. Wackrow, and J. H. Chandler. Automatic detection of blurred images in UAV image sets. *ISPRS Journal of Photogrammetry and Remote Sensing*, 122: 1–16, 2016. doi: 10.1016/j.isprsjprs.2016.09.010.

[56] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2015. Published at ICLR 2015.

[57] M. Sivakumar and T. Y. J. Naga Malleswari. A literature survey of unmanned aerial vehicle usage for civil applications. *Journal of Aerospace Technology and Management*, 13:1–23, 2021. doi: 10.1590/jatm.v13.1233.

[58] G. G. Stokes. On the theory of oscillatory waves. *Transactions of the Cambridge Philosophical Society*, 8:441–455, 1847. Reprinted in *Mathematical and Physical Papers*, Volume I, Cambridge University Press, 1880, pp. 197–229.

[59] J. Tessendorf. Simulating ocean water. In *SIGGRAPH Course Notes (Course 47: Simulating Nature: Realistic and Interactive Techniques)*. ACM Press, 2004. Originally presented 1999; revised through 2004. Available at `https://people.computing.clemson.edu/~jtessen/reports/papers_files/coursenotes2004.pdf`.

[60] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30, 2017. doi: 10.1109/IROS.2017.8202133.

[61] C. Tomasi and T. Kanade. Detection and tracking of point features. Technical Report CMU-CS-91-132, Carnegie Mellon University, 1991.

[62] J. Tremblay, A. Prakash, D. Acuna, M. Brober, V. Jampani, C. Anil, T. To, E. Cameracci, S. Boochoon, and S. Birchfield. Training deep networks with synthetic data: Bridging the reality gap by domain randomization. In *2018 IEEE/CVF Conference*

on Computer Vision and Pattern Recognition Workshops (CVPRW), pages 969–977, 2018. doi: 10.1109/CVPRW.2018.00143.

[63] R. Wang, G. Ma, Q. Qin, Q. Shi, and J. Huang. Blind UAV images deblurring based on discriminative networks. *Sensors*, 18(9):2874, Aug. 2018. doi: 10.3390/s18092874. URL `https://doi.org/10.3390/s18092874`.

[64] O. Whyte, J. Sivic, A. Zisserman, and J. Ponce. Non-uniform deblurring for shaken images. *International Journal of Computer Vision*, 98(2):168–186, 2012. doi: 10.1007/s11263-011-0502-7.

[65] D. Wierzbicki. Multi-camera imaging system for uav photogrammetry. *Sensors*, 18 (8), 2018. ISSN 1424-8220. doi: 10.3390/s18082433. URL `https://www.mdpi.com/1424-8220/18/8/2433`.

[66] T. N. Wolf, S. Pölsterl, and C. Wachinger. DAFT: A universal module to interweave tabular data and 3D images in CNNs. *NeuroImage*, 260:119505, 2022. doi: 10.1016/j.neuroimage.2022.119505.

[67] X. Wu, W. Li, D. Hong, R. Tao, and Q. Du. Deep learning for unmanned aerial vehicle-based object detection and tracking: A survey. *IEEE Geoscience and Remote Sensing Magazine*, 10(1):91–124, 2022. doi: 10.1109/MGRS.2021.3115137.

[68] Q. xing Zhang, G. hua Lin, Y. ming Zhang, G. Xu, and J. jun Wang. Wildland forest fire smoke detection based on faster r-cnn using synthetic smoke images. *Procedia Engineering*, 211:441–446, 2018. ISSN 1877-7058. doi: https://doi.org/10.1016/j.proeng.2017.12.034. URL `https://www.sciencedirect.com/science/article/pii/S1877705817362574`. 2017 8th International Conference on Fire Science and Fire Protection Engineering (ICFSFPE 2017).

[69] C.-Y. Xiong, L. Yao, and J.-X. Shen. Texture classification based on EMD and FFT. *Journal of Zhejiang University-SCIENCE A*, 7:1516–1521, 2006. doi: 10.1631/jzus.2006.A1516.

[70] H. Xu, B. Jiang, W. Li, M. Zhu, Z. Li, T. Pang, M. Gao, and S. Chen. A novel formula for micro-UAV swarm systems: Architecture, algorithms, and verification. *Digital Communications and Networks*, 11(5):1543–1553, 2025. doi: 10.1016/j.dcan.2025.07.007.

[71] Y. Yang, J. Zhang, W. Huang, and T. Xie. Roughness analysis of sea surface from visible images by texture. *IEEE Access*, 8:46448–46458, 2020. doi: 10.1109/ACCESS.2020.2978115.

[72] H. C. Yuen and B. M. Lake. Nonlinear dynamics of deep-water gravity waves. volume 22 of *Advances in Applied Mechanics*, pages 67–229. Elsevier, 1982. doi: https://doi.org/10.1016/S0065-2156(08)70066-8. URL `https://www.sciencedirect.com/science/article/pii/S0065215608700668`.

[73] X. Zhang, Z. He, Z. Ma, P. Jun, and K. Yang. Viae-net: An end-to-end altitude estimation through monocular vision and inertial feature fusion neural networks for uav autonomous landing. *Sensors*, 21(18):6302, 2021. doi: 10.3390/s21186302.

[74] F. Zhao, C. Zhang, and B. Geng. Deep multimodal data fusion. *ACM Computing Surveys*, 56(9):1–36, 2024. doi: 10.1145/3649447.